

119th Doctoral's Dissertation
Thesis Advisor : Eunwoo Kim

**Toward Adaptive Robot Learning
with Foundation Models:
From Task Planning to Action Policies**

August 2026

**The Graduate School
Chung-Ang University
Major in Big Data
Department of Computer Science and Engineering
Jiho Lee**

**Toward Adaptive Robot Learning
with Foundation Models:
From Task Planning to Action Policies**

**Presented to the Faculties of the Chung-Ang University
in Partial Fulfillment of the Requirement
of the Degree of Doctor of Philosophy**

August 2026

**The Graduate School
Chung-Ang University
Major in Big Data
Department of Computer Science and Engineering
Jiho Lee**

**Toward Adaptive Robot Learning
with Foundation Models:
From Task Planning to Action Policies**

By
Jiho Lee

Department of Computer Science and Engineering
Chung-Ang University

Date: _____

Approved:

Junseok Kwon

Eunwoo Kim

Youngbin Kim

Jongwon Choi

Bumsoo Kim

Dissertation submitted in partial fulfillment of the
requirement for the degree of Doctor of Philosophy in
the Department of Computer Science and Engineering
in the Graduate School of Chung-Ang University.

ABSTRACT

Jiho Lee

Major in Big Data

Department of Computer Science and Engineering

Chung-Ang University Graduated School

Recent advances in robot learning have been driven by foundation models trained on large-scale data with language-grounded representations, enabling strong generalization in both high-level task planning and low-level action policies. However, real-world deployment requires more than the ability to generalize from pretrained knowledge. Robots must operate in a dynamic physical world where perceptual conditions, interaction dynamics, and task requirements can continually change. In such environments, pretrained knowledge does not always translate into robot behavior appropriate for the current context. This dissertation views this limitation as a knowledge-to-behavior mismatch, in which the knowledge encoded in foundation models fails to produce reliable, feasible, or context-appropriate robot behavior.

To mitigate this mismatch, this dissertation proposes an adaptive robot learning framework based on foundation models. The central idea is that robots should not merely rely on static pretrained knowledge, but should continually improve their behavior by adapting that knowledge according to the source of mismatch they encounter across changing deployment conditions. Specifically, this dissertation focuses on four complementary directions of knowledge adaptation: acquiring useful knowledge from experience when pretrained knowledge is insufficient, removing harmful knowledge that leads to unreliable behavior, refining uncertain knowledge, and aligning situated knowledge with the current context. This dissertation instantiates these four directions of knowledge adaptation at the levels most directly involved in robot behavior generation, including task planning and action policies.

At the level of high-level task planning, this dissertation develops active knowledge acquisition and self-corrective knowledge refinement methods for large language model-based task planners. Although large language model-based planners provide strong reasoning capabilities, their reliance on

pretrained knowledge limits their ability to incorporate new experiences into learning and revise erroneous reasoning during execution. To address insufficient knowledge, active knowledge acquisition through contrastive in-context learning enables the model to acquire useful knowledge from experience by contrasting successful and failed experiences stored in active memory. To address uncertain knowledge, self-corrective knowledge refinement through feedback-guided inverse reasoning enables the model to validate, analyze, and revise its own reasoning process by backward verification.

At the level of low-level action policies, this dissertation develops selective knowledge removal and environment-aware knowledge alignment methods for vision-language-action models. Although vision-language-action models provide strong generalization by grounding language instructions in visual observations and robot actions, they may still produce unreliable behavior under unseen visual and physical conditions. To address harmful knowledge, selective knowledge removal through policy unlearning suppresses internal activation patterns associated with unreliable behavior while preserving representations necessary for successful task execution. To address situated knowledge, environment-aware knowledge alignment through test-time adaptation realigns visual and action representations disrupted by deployment shifts using a trajectory collected at test time.

Overall, this dissertation establishes an adaptive robot learning framework based on foundation models for mitigating knowledge-to-behavior mismatch. By enabling robots to acquire useful knowledge from experience, remove harmful knowledge, refine uncertain decisions, and align their behavior with unseen conditions, this dissertation provides a step toward robotic systems that can continually improve, generalize reliably, and operate robustly in dynamic real-world environments.

Keywords: *Robot Learning, Foundation Models, Large-Language Models, Vision-Language-Action Models, Active Learning, Machine Unlearning, Self-Correction, Test-Time Adaptation*

Contents

ABSTRACT	i
List of Tables	vi
List of Figures	viii
1 Introduction	1
1.1 Main Challenges	3
1.2 Research Objectives	6
1.3 Organization of Dissertation	8
2 Background	12
2.1 Large Language Models	12
2.2 Vision Language Action Models	14
3 Active Knowledge Acquisition through Contrastive Learning	16
3.1 Related Works	20
3.1.1 Memory-Based Planning	20
3.2 Proposed Method	21
3.2.1 Overview	21
3.2.2 Active Memory	25
3.2.3 Contrastive In-Context Learning with Dual Prompting	26
3.3 Experiments	27
3.3.1 Benchmark Experiments	27
3.3.2 Real-World Experiments	37
3.4 Discussion	39

4	Selective Knowledge Removal through Policy Unlearning	40
4.1	Related Works	43
4.1.1	Machine Unlearning	43
4.1.2	Sparse Autoencoders	43
4.2	State Hallucination in Vision Language Action Models	44
4.2.1	Definition	44
4.2.2	Characterization	45
4.2.3	Attention Analysis	46
4.2.4	Mechanistic Analysis	48
4.3	Proposed Method	51
4.3.1	Region-Guided Feature Selection	51
4.3.2	Feature-Guided Policy Unlearning	52
4.4	Experiments	53
4.4.1	Benchmark Experiments	53
4.4.2	Real-World Experiments	60
4.5	Discussion	61
5	Self-Corrective Knowledge Refinement through Feedback-Guided Inverse Reasoning	63
5.1	Related Works	67
5.1.1	Corrective Planning	67
5.2	Proposed Method	69
5.2.1	Framework	69
5.2.2	Inverse Prompting	71
5.3	Experiments	73
5.3.1	Benchmark Experiments	73
5.3.2	Real-World Experiments	79
5.4	Discussion	83
6	Environment-Aware Knowledge Alignment through Test-Time Adaptation	84
6.1	Related Works	90
6.1.1	Test-Time Strategies	90
6.2	Proposed Method	92

6.2.1	Problem Statement: Test-Time Adaptation of Vision Language Action Models	92
6.2.2	TTA-VLA: Test-time Trajectory-based Adaptation for Vision Language Action models	94
6.3	Experiments	98
6.3.1	Benchmark Experiments	98
6.3.2	Real-World Experiments	108
6.4	Discussion	116
7	Conclusion	117
7.1	Summary	117
7.2	Future Directions	120
	References	122
	국 문 초 록	139

List of Tables

3.1	Comparison results of task success rates for active knowledge acquisition.	32
3.2	Ablation study in terms of active memory, dual prompting, and selector.	34
3.3	Analysis of prompt ratio for active knowledge acquisition. . .	36
4.1	Performance comparison for selective knowledge removal. . .	55
4.2	Comparison between global, tokenized, and region-guided feature unlearning.	57
4.3	Ablation study of score aggregation and the retain objective.	59
5.1	Comparison results of task success rates for self-corrective knowledge refinement.	76
5.2	Performance of the self-correction process.	77
6.1	Comparison results of task success rates on benchmark experiments for environment-aware knowledge alignment.	101
6.2	Performance improvements with projection-based alignment compared to angular-based alignment.	105
6.3	Comparison results of task success rates on real-world experiments for environment-aware knowledge alignment.	112

List of Figures

1.1	Conceptual overview of knowledge-to-behavior mismatch. . .	3
1.2	Organization of dissertation around four directions of knowledge adaptation.	8
3.1	Graphical illustration of in-context learning methods for task planning.	17
3.2	Examples of positive and negative prompting.	19
3.3	Graphical illustration of LLM-based task planning approaches.	24
3.4	Comparison of task success rate and memory size for active knowledge acquisition.	35
3.5	Comparison of execution sequences in the real-world experiments.	38
4.1	Graphical illustrations of how the proposed sparse feature unlearning mitigates hallucinated beliefs in VLAs.	42
4.2	Definition and examples of state hallucination in VLAs. . . .	45
4.3	Examples and statistics of state hallucination in VLA execution.	46
4.4	Quantitative comparison of event-aligned attention patterns between normal and hallucination executions.	47
4.5	Visualization of hallucination- and clean-relevant sparse features.	50
4.6	Overall framework of the proposed selective knowledge removal method.	51
4.7	Comparison of rollouts and feature activations before and after unlearning on the benchmark.	56
4.8	Preservation and improvement analysis of the proposed unlearning method.	58
4.9	Comparison of rollouts and feature activations before and after unlearning on the real robot.	61

5.1	Graphical illustration of LLM-based task planning methods in terms of correction.	64
5.2	Examples of the generated feedback with the standard prompting and the proposed inverse prompting.	65
5.3	Graphical illustration of self-corrective knowledge refinement process.	70
5.4	Comparison of correction attempts.	78
5.5	Comparison of execution sequences in real-world scenarios.	82
6.1	Examples of various visual perturbation types in LIBERO-Plus.	85
6.2	Comparison of VLAs under in-distribution and out-of-distribution environments.	87
6.3	Graphical illustration of environment-aware knowledge alignment.	95
6.4	Comparison of task success rates according to task difficulty under visual perturbations.	102
6.5	Knowledge preservation and improvement under various visual perturbations compared to the original policy.	103
6.6	Visualizations of rollouts with the original policy and proposed method on benchmark experiments.	107
6.7	Examples of real-world task environments under different visual perturbations for the task of Pick-and-place.	109
6.8	Comparison of real-world rollouts between the original policy and the proposed method.	114
6.9	Visualizations of failed real-world rollouts for environment-aware knowledge alignment.	115

1. Introduction

Robots are becoming an indispensable part of modern society, from daily life and workplaces to large-scale industrial environments. As robots are deployed in increasingly diverse and complex settings, they are expected not only to execute predefined motions, but also to understand their surroundings, interpret human instructions, reason about task goals, and adapt their behaviors to new and changing situations. To support these capabilities, robot learning has emerged as a central paradigm that moves beyond traditional rule-based robotic systems by enabling robots to acquire task knowledge and improve their behaviors from data and experience.

Robot learning requires decision-making at different levels, from high-level task planning to low-level action policies. At a higher level, robots need to determine what should be done. They interpret language instructions, understand task goals, and organize actions into coherent plans. This level is commonly referred to as task planning. At a lower level, robots need to determine how the planned behavior should be physically executed. They map observations and task instructions to motor actions. This level corresponds to action policy. Recent advances in foundation models, including large language models (LLMs), vision-language models (VLMs), and vision-language-action models (VLAs), have significantly expanded the capabilities

of robot learning across these levels. By leveraging large-scale pretraining and multi-modal representations, these models enable robots to interpret natural-language instructions, generalize across diverse tasks, and execute a wide range of embodied behaviors with minimal task-specific supervision.

However, real-world deployment requires more than the ability to generalize from pretrained knowledge. Robots must operate in dynamic physical environments where visual appearances, object configurations, interaction dynamics, and task requirements can continuously change. In such environments, pretrained knowledge does not always translate into robot behavior that is appropriate for the current context. This dissertation views this limitation as a knowledge-to-behavior mismatch, where the knowledge encoded in foundation models fails to produce reliable, feasible, or context-appropriate robot behavior. Mitigating this mismatch requires robots to adapt pretrained knowledge according to the source of mismatch they encounter, enabling their behavior to continually improve under changing deployment conditions.

Figure 1.1 illustrates the overall concept of knowledge-to-behavior mismatch considered in this dissertation. As robots interact with continuously changing environments, this mismatch can arise from different characteristics of pretrained knowledge, including insufficient, harmful, uncertain, and situated knowledge. Knowledge may be insufficient when it lacks the information needed to handle novel situations, harmful when it induces unreliable or undesirable behavioral consequences, uncertain when its validity or

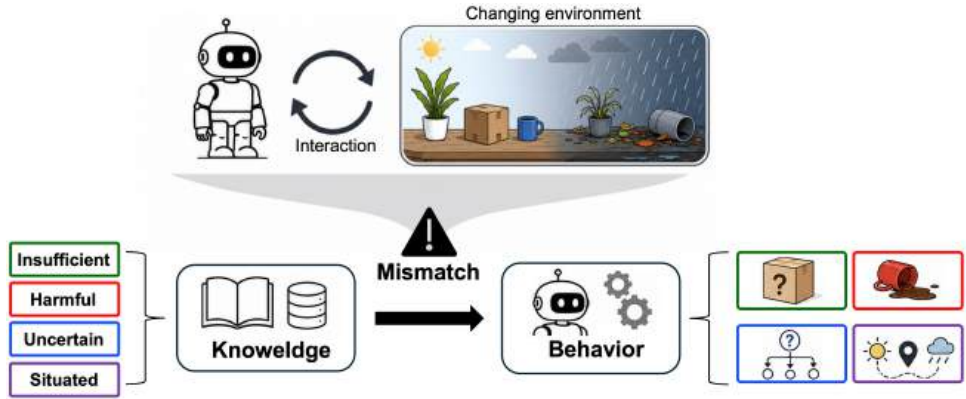


Figure 1.1: Conceptual overview of knowledge-to-behavior mismatch considered in this dissertation. Pretrained knowledge may be insufficient, harmful, uncertain, or situated, preventing robots from generating appropriate behavior in changing environments.

applicability has not been sufficiently verified, and situated when its feasibility depends on a specific environment or interaction context. These representative characteristics provide the basis for organizing the main challenges addressed in this dissertation.

1.1 Main Challenges

Building on the knowledge-to-behavior mismatch discussed above, this section summarizes the main challenges addressed in this dissertation. These challenges arise when foundation-model-based robots use pretrained knowledge to generate reliable behavior across high-level task planning and low-level action policies under changing environments.

- **Passive knowledge acquisition.** When pretrained knowledge is insufficient, robots need to acquire additional knowledge from experi-

ence gained through their own interactions. However, most existing data-driven robot learning methods acquire knowledge in a largely passive manner by relying on pre-collected datasets or manually curated demonstrations for training. In contrast, robots can continuously generate diverse experiences through interaction with their environments. These experiences may include successful executions, failed attempts, partial successes, and variations in task conditions, each of which can provide different learning signals. However, existing methods often lack mechanisms to process such continuously generated experiences and actively determine which ones should be collected, retained, or used for learning. As a result, potentially informative experiences remain underutilized, limiting the ability of robots to actively acquire useful knowledge from their own interactions.

- **Persistence of harmful knowledge.** When pretrained knowledge is associated with unreliable or undesirable behavior, robots need mechanisms to reduce its influence on action generation. Conventional robot learning pipelines primarily focus on reinforcing knowledge from successful demonstrations. However, this success-biased learning process provides limited mechanisms for explicitly reducing knowledge associated with unreliable behaviors. As a result, such knowledge may persist in the policy and cause the robot to execute behaviors that are inconsistent with the physical state of the environment. In particular, failed or misleading experiences should not be treated merely as less

useful data for learning. Instead, they can provide important signals for identifying knowledge that should be reduced, avoided, or removed from the policy. Nevertheless, existing methods often lack mechanisms to selectively identify and unlearn harmful knowledge while preserving useful task knowledge for task execution.

- **Lack of self-corrective knowledge refinement.** When the validity or applicability of pretrained knowledge is uncertain, robots need to verify and refine their reasoning before generating executable behavior. Robotic reasoning requires decisions that are consistent with physical constraints, causal relationships, and the sequential structure of tasks. However, reasoning based on foundation models is not always grounded in these requirements, often resulting in hallucinated, incomplete, or logically inconsistent decisions. Existing robot learning approaches often lack mechanisms that allow models to use feedback to detect, analyze, and refine their own reasoning errors. As a result, robots may rely on external validators, human intervention, or hand-engineered rules, rather than autonomously refining unverified reasoning through a self-corrective process.
- **Environmental dependency of situated knowledge.** When pre-trained knowledge is situated, its feasibility depends on the environment or interaction context in which it is deployed. During deployment, robot policies often encounter conditions that differ from those

observed during training, including changes in visual appearance, sensor inputs, object configurations, and physical dynamics. These distribution shifts can degrade performance by disrupting the relationship between pretrained knowledge and observations from the deployment environment. In this case, the knowledge used by the policy is not necessarily absent or harmful, but insufficiently aligned with the current context. However, most existing methods keep their knowledge fixed during deployment and do not exploit observations from the new environment as adaptation signals. As a result, robot policies may fail to align situated knowledge with unseen environments, limiting their robustness in diverse real-world conditions.

1.2 Research Objectives

The challenges discussed above raise the question of what kind of learning process can enable robots to continually improve their behavior by adjusting how pretrained knowledge is used, updated, and grounded for behavior generation in changing environments. Human adaptive learning provides a useful conceptual basis for addressing this question. In everyday life, humans continually improve how they act in the world by learning from information, experience, feedback, consequences, and context. They are continuously exposed to vast amounts of information and experience, but they do not treat all information as equally useful. Instead, they selectively acquire knowledge that is relevant to their goals or preferences, refine their under-

standing through experience and feedback, avoid or suppress ineffective behaviors based on their consequences, and adjust their decisions according to the current context. This form of adaptive self-improvement suggests that reliable behavior requires the ability to extend, revise, suppress, and align prior knowledge according to experience, feedback, consequences, and context.

Building on this view of human adaptive learning, this dissertation proposes an adaptive robot learning framework based on foundation models. In this dissertation, adaptability refers to the ability to adjust how pretrained knowledge supports reliable behavior generation in response to changing interaction experiences, feedback, and environmental conditions. The objective of this dissertation is to mitigate knowledge-to-behavior mismatch by developing adaptive learning mechanisms that enable robots to continually improve their behavior by acquiring useful knowledge, removing harmful knowledge, refining uncertain knowledge, and aligning situated knowledge for reliable behavior generation.

Specifically, this dissertation investigates four complementary capabilities for adaptive robot learning. First, active knowledge acquisition addresses insufficient knowledge by enabling robots to acquire useful knowledge from their own interaction experiences. Second, selective knowledge removal addresses harmful knowledge by suppressing knowledge associated with unreliable behavior. Third, self-corrective knowledge refinement addresses uncertain knowledge by improving the reliability of reasoning. Fourth, environment-

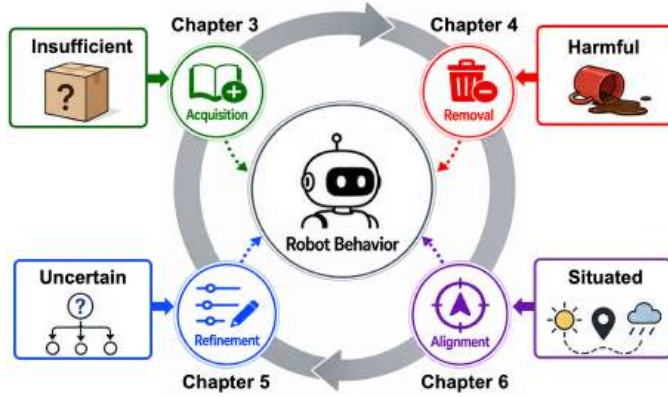


Figure 1.2: Organization of dissertation around four directions of knowledge adaptation. Each chapter addresses a different source of knowledge-to-behavior mismatch: insufficient knowledge through active knowledge acquisition (Chapter 3), harmful knowledge through selective knowledge removal (Chapter 4), uncertain knowledge through self-corrective knowledge refinement (Chapter 5), and situated knowledge through environment-aware knowledge alignment (Chapter 6).

aware knowledge alignment addresses situated knowledge by adapting pre-trained knowledge to the current deployment environment. Together, these directions aim to support behavioral self-improvement, enabling more reliable robot behavior across high-level task planning and low-level action policy in dynamic real-world environments.

1.3 Organization of Dissertation

To achieve these objectives, this dissertation is organized around robot learning with foundation models from task planning to action policy execution. Chapter 2 first reviews background literature on robot learning with foundation models, focusing on LLM-based task planning and VLA-based

action policies. Building on this background, the remaining chapters are organized around four directions of knowledge adaptation, as illustrated in Figure 1.2.

Each direction addresses a different source of knowledge-to-behavior mismatch and is implemented at the level where it can most directly influence robot behavior. At the task-planning level, active knowledge acquisition and self-corrective knowledge refinement are studied with LLM-based task planners in Chapters 3 and 5, respectively. These directions focus on the upstream decision-making process, where acquired experience and refined reasoning guide how the robot interprets tasks and composes action plans. At the action-policy level, selective knowledge removal and environment-aware knowledge alignment are studied with VLA-based action policies in Chapters 4 and 6, respectively. These directions focus on the policy execution process, where harmful internal representations or environment-dependent representations can directly affect generated robot actions.

Together, this organization highlights how foundation models can support adaptive robot learning by mitigating knowledge-to-behavior mismatch across both task planning and action policy. The following chapters describe how each direction is instantiated as a concrete learning mechanism for adaptive robot learning with foundation models.

Chapter 3 presents a contrastive learning framework with an active memory mechanism for LLM-based task planning. This chapter investigates active knowledge acquisition, where robots acquire useful task knowledge from

generated experiences instead of passively relying on fixed datasets. The proposed method uses a diversity-aware active memory mechanism to retain informative examples within a limited memory budget. By contrasting successful and failed experiences in active memory with positive and negative prompting, the method reinforces useful reasoning patterns while helping the model recognize and avoid incorrect behaviors.

Chapter 4 introduces a policy unlearning framework for VLA-based action policies. This chapter focuses on selective knowledge removal, where harmful policy knowledge is reduced while useful task knowledge is preserved. The proposed method analyzes internal representations of the policy to distinguish hallucination-related activation patterns from representations that support successful task execution. Based on this analysis, it suppresses activation patterns associated with unreliable behavior while preserving representations necessary for successful task execution.

Chapter 5 proposes a self-corrective task planning method based on feedback-guided inverse reasoning for LLM-based task planning. This chapter focuses on self-corrective knowledge refinement, where the model detects, analyzes, and revises its own erroneous reasoning. The method incorporates a multi-step reasoning process that generates clear and interpretable feedback during plan generation. By generating inverse actions and verifying whether they can restore the system to its original state, the method evaluates the logical coherence of generated plans and refines erroneous decisions using self-generated feedback, thereby supporting autonomous plan correc-

tion.

Chapter 6 introduces a test-time adaptation framework for VLA-based action policies. This chapter focuses on environment-aware knowledge alignment, where pretrained policy knowledge is adapted to unseen deployment environments. The proposed method uses a trajectory collected at test time to realign disrupted temporal visual-action relationships under distribution shifts. It further anchors the adaptation process to natural-language task instructions as an environment-invariant reference, preserving semantic stability while mitigating representational drift.

Finally, Chapter 7 concludes this dissertation and discusses future research directions toward more adaptive and reliable robot learning with foundation models.

2. Background

This chapter introduces the background on robot learning with foundation models that underlies this dissertation. We focus on two major directions. The first direction is large language models for robot task planning, where they are used to generate high-level action sequences from task instructions, contextual examples, and environmental information. The second direction is vision language action models for robot control, where multimodal models directly map visual observations and language instructions to robot actions. This overview establishes the foundation for the following chapters, which discuss more specific lines of work in relation to each proposed method.

2.1 Large Language Models

Large language models (LLMs) have been increasingly utilized for robot task planning due to their ability to handle complex sequential reasoning with rich commonsense knowledge [1–5]. To leverage pretrained LLMs without additional parameter updates, in-context learning [6] has emerged as a powerful learning strategy. By conditioning the model on a small set of input-output examples, LLMs can rapidly adapt to new tasks and generate contextually appropriate responses. This capability has recently been

applied to robotic task planning [1–4, 7–18], where contextual examples facilitate the generation of coherent and executable action sequences, demonstrating their flexibility and generalization ability.

Most LLM-based planning approaches generate task plans by grounding language instructions in environmental information, predefined robot skills, or symbolic planning representations [1–4, 8, 9]. For example, some approaches combine LLM-based task grounding with affordance functions to account for the current environment and predefined skills [1]. Other approaches use scene descriptors together with visual question answering models [19] or human-provided scene descriptions to obtain environment feedback [2]. LLM-based translators and planners have also been proposed to convert natural-language instructions into Planning Domain Definition Language (PDDL) formulations [3], where PDDL provides a symbolic representation of planning domains and problems [20, 21]. In addition, LLM-based planners have been used to generate action sequences while automatically producing trainable datasets from language instructions without human supervision [4]. Partially observable task planning frameworks further extend LLM-based planning by considering uncertainties in the environment and skill execution [8].

In addition, some works generate executable code representations of task plans [10, 11, 13]. Programmatic prompting templates have been introduced to generate structured robot-executable code [10, 13]. Multi-level task decomposition has also been developed for multi-robot frameworks using code-

style representations [11].

These studies demonstrate the potential of LLMs as high-level robot planners, but they also highlight that reliable task planning requires mechanisms for actively acquiring useful task knowledge from experiences and refining uncertain knowledge. These challenges motivate the studies presented in Chapter 3 and Chapter 5.

2.2 Vision Language Action Models

Built upon the strong compositionality and semantic grounding capabilities of pretrained vision-language models (VLMs) [22–24], vision-language-action models (VLAs) have recently emerged as a major framework for generalist robot control. Typically, VLAs take sensory observations and language instructions as input and produce low-level robot actions as output [25, 26]. This formulation enables robot policies to ground action generation in both perceptual observations and language-conditioned task semantics, allowing robots to perform diverse manipulation tasks across different objects, scenes, and instructions.

Recent VLAs improve generalization by scaling pretrained vision-language representations, robot demonstration data, and action-generation architectures [27–34]. These architectures can be broadly categorized into autoregressive policies that extend VLMs to sequential action generation [27, 28, 33, 35, 36], diffusion-based methods that produce continuous actions through iterative denoising [29, 37–40], and CLIP-based approaches that use simi-

larities between visual contexts and language-based motion primitives [41]. Alongside these architectural developments, large-scale multimodal robot datasets have played an important role in improving the coverage and diversity of tasks, objects, and environments [28–30, 41–44].

More recently, several studies have augmented VLAs with additional reasoning, recovery, or inference-time selection mechanisms. Some methods incorporate high-level reasoning mechanisms, such as subtask-conditioned action generation or intermediate goal prediction [31, 43, 45, 46, 46–49]. Other methods improve deployment reliability by detecting and correcting execution failures [50, 51] or by sampling and verifying multiple candidate actions at inference time [52, 53].

These studies establish VLAs as a promising framework for generalist robot control, but they also highlight the need for mechanisms that selectively reduce unreliable behaviors for reliability and align learned representations with changing deployment environments for adaptability. These challenges motivate the studies presented in Chapter 4 and Chapter 6.

3. Active Knowledge Acquisition through Contrastive Learning

Large language models (LLMs) have shown impressive generalization and reasoning capabilities across a wide range of tasks, from natural language understanding [54] to code generation [55], by leveraging extensive pre-trained knowledge. Recently, these capabilities have been extended to robotic task planning [1–4, 7–12], where an LLM generates action sequences through in-context learning [6]. In this approach, the model is provided with a few input-output examples (referred to as contextual memory) and generates plans without requiring additional fine-tuning. It enables task planning in scenarios with varying task specifications [1, 2, 7].

Most existing works employ contextual memory for in-context learning, typically composed of successful demonstrations manually curated by human experts [1–3, 7, 13], as illustrated in Figure 3.1(a). These approaches primarily focus on positive examples, which guide the model toward ideal action sequences. However, they overlook valuable learning signals from *negative examples*, which can help the model recognize and avoid incorrect behaviors [56].

In LLM-based task planning, it is challenging to generate demonstrations

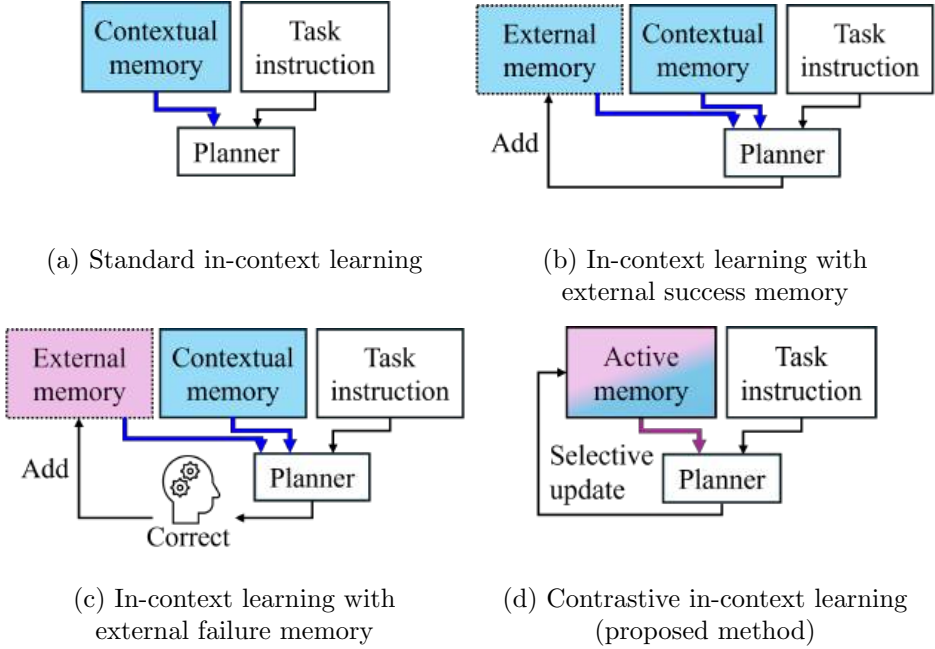


Figure 3.1: Graphical illustration of in-context learning methods for task planning. In memory, blue and purple colors represent examples derived from successful and failed demonstrations, respectively. Memories with dashed and solid borders indicate growing and fixed sizes, respectively. (a) learns with positive examples from a pre-defined contextual memory. (b) conducts in-context learning using successful demonstrations stored in an external memory, and (c) learns from corrected examples derived from failures in an external memory. In contrast, ours in (d) performs contrastive in-context learning with both successful and failure demonstrations by selectively incorporating them into an active memory of a limited size during task planning.

that support learning from positive and negative examples. This is because such demonstrations require complex reasoning over temporal and causal dependencies to achieve a given goal, typically in a structured language-based representations, such as symbolic actions [3, 13] or planning domain languages [20, 21]. This leads to error-prone human-crafted examples, especially in long-horizon tasks with increased planning complexity. To ad-

dress this, we utilize demonstrations generated by LLMs during task planning, where the LLM outputs the structured action sequences. Recently, a few works have leveraged generated demonstrations by storing them in an external memory for future knowledge retrievals when executing new task instructions [14–16]. They either collect only successful demonstrations [14] or corrected demonstrations from failures [15, 16], both of which also provide the model with positive examples, as in Figure 3.1(b) and 3.1(c). However, they lead to unbounded memory expansion and inefficient retrieval due to an extensive number of generated examples in the memory. Moreover, they often require an additional step to store or transform specific types of knowledge, such as for LLM querying.

To address these problems, we propose a novel approach that introduces *contrastive in-context learning* for task planning, utilizing both successful and failure demonstrations through a dual prompting strategy. This method employs two prompts to learn by exploiting the contrast between them, where positive prompts derived from successes guide desirable reasoning patterns, and negative prompts from failures help in recognizing incorrect behaviors, as depicted in Figure 3.2. To make use of both types of demonstrations, we introduce an active memory that serves as a self-evolving repository of LLM-generated demonstrations to provide task-specific knowledge while selectively integrating useful ones. Additionally, we adopt a diversity-aware selection strategy to ensure a compact yet informative memory, identifying a representative set of demonstrations. In doing so, the robot can continu-

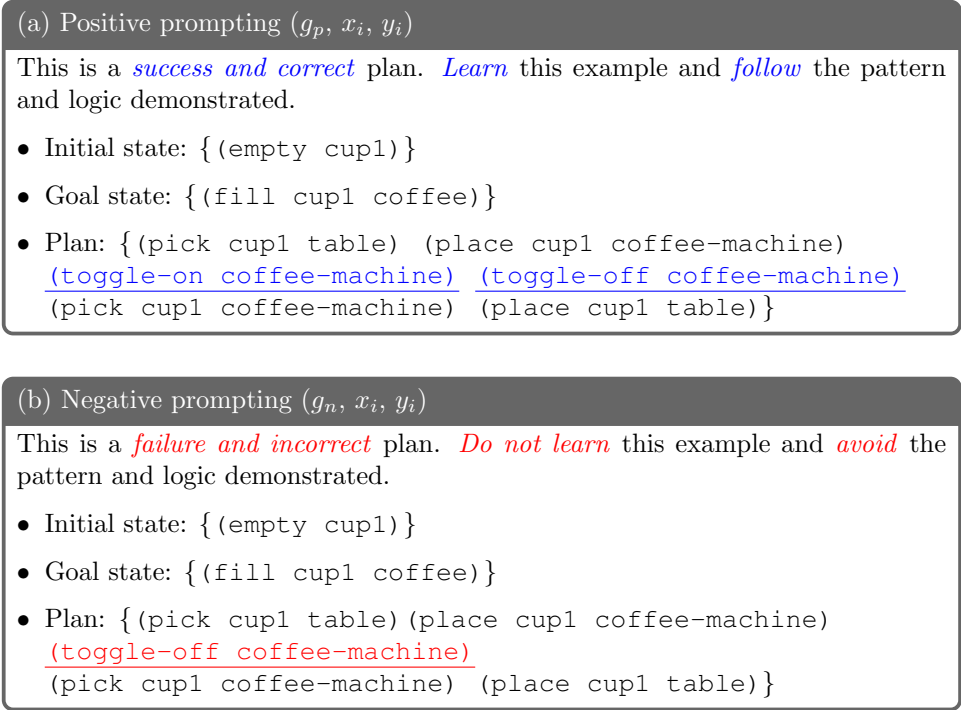


Figure 3.2: Examples of positive and negative prompting. (a) Positive prompting helps the model learn correct behaviors through an instruction and a successful demonstration, while (b) negative prompting encourages avoiding incorrect behaviors with a negative learning signal and a failed demonstration. Underlined texts indicate the differences between the two demonstrations.

ally improve its planning ability by distilling its own experiences, including successes and failures, without requiring additional supervision or resources.

To validate the effectiveness of our method, we conduct evaluations on a diverse set of benchmark scenarios, each reflecting key challenges in LLM-based robot task planning: Shelf [57] for multi-object manipulation, Coffee [7] for long-horizon task sequences, and Unpacking [58] for spatially con-

strained tasks. Experimental results demonstrate that the proposed contrastive in-context learning approach significantly outperforms existing in-context learning based task planning methods, improving task success rates by up to 20.8% with an average gain of 10.7%, while reducing memory usage by up to 75.0%. Furthermore, we demonstrate the real-world applicability of our approach, showing that leveraging both successful and failed cases through contrastive in-context learning helps prevent potential mistakes, compared to the standard in-context learning, which relies on pre-defined successful examples.

3.1 Related Works

3.1.1 Memory-Based Planning

Although in-context learning has been widely used for LLM-based robot task planning, many approaches rely on manually curated successful examples in a pre-defined contextual memory, limiting the ability of the model to learn from diverse examples. To address this limitation, recent works have explored memory-augmented approaches that incorporate demonstrations generated by LLMs during task planning into an external memory and reuse them as contextual examples [14–16]. These methods aim to improve planning by expanding the set of examples available to the LLM. One line of work collects successful experiences generated during task planning and reuses them as contextual examples [14]. Other methods store transformed

feedback or improved plans obtained by querying an LLM with failed outputs [15, 16].

However, existing memory-augmented approaches have several limitations. First, they mainly rely on positive prompting, where the model is encouraged to imitate successful or corrected examples. Second, memory can grow continuously as more experiences are collected, leading to resource overhead and inefficient retrieval. Third, these methods typically store only a subset of experiences, such as successful demonstrations [14] or corrections derived from failures [15, 16], which limits the diversity of learned knowledge. In contrast, we propose contrastive in-context learning with dual prompting, which provides both positive and negative examples to guide the model on what to imitate and what to avoid. To realize this in a resource-constrained robotic system, we introduce an active memory module that continuously curates and updates a compact collection of successful and failed LLM-generated demonstrations, selectively retaining the most useful examples within a limited memory budget.

3.2 Proposed Method

3.2.1 Overview

Our goal is to enable large language models (LLMs) to enhance their learning capabilities by fully utilizing both self-generated successful and failed experiences under a limited memory budget. Typically, existing works [1,

2,10,13] perform standard in-context learning using a fixed contextual memory, M_F , manually constructed by human experts, as illustrated in Figure 3.3(a). $M_F = \{d_1, \dots, d_B\}$ is a set of B successful demonstrations, where each $d_i = (x_i, y_i)$ consists of an input-output pair that encourages the model to imitate the provided behaviors as positive prompting. To broaden the learning capability using self-generated new experiences during task planning, recent works have attempted to store them in an external memory by transforming and accumulating them [14–16], as depicted in Figure 3.3(b) (transforming can be omitted). However, they indiscriminately accumulate data into an unbounded memory, hindering efficient retrieval, and often require additional steps to store or transform specific types of knowledge for positive prompting [15, 16].

To address these limitations, we introduce a contrastive in-context learning framework with an active memory to effectively leverage both generated successful and failed knowledge during task planning. The framework employs a dual prompting strategy that contrasts positive prompts from successes and negative prompts from failures to allow the model to imitate correct behaviors and avoid mistakes. As shown in Figure 3.3(c), the planner generates an action sequence as a demonstration for the given task instruction, resulting in success or failure. A selector module evaluates existing and newly generated demonstrations by estimating their utility using a diversity-aware approach in the language feature space. It updates the active memory by selecting the most informative subset while preserving the fixed capacity

of M_F . The updated active memory is then used to provide contrastive examples during in-context learning, enabling an LLM to continuously refine its planning ability with a compact yet diverse set of experiences.

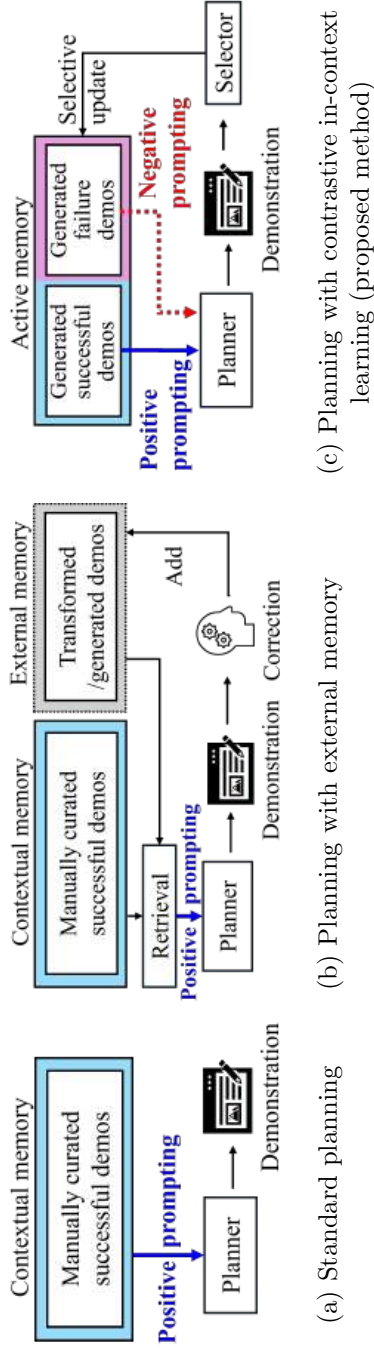


Figure 3.3: Graphical illustration of LLM-based task planning approaches. (a) generates a demonstration as an action sequence by performing positive prompting with manually curated successful demonstrations to guide the planner toward optimal planning behaviors. (b) conducts positive prompting using an external memory that retains successful demonstrations or refined demonstrations obtained from the correction of failures. In contrast, (c) employs positive and negative prompting by utilizing generated successful and failed demonstrations stored in an active memory, respectively.

3.2.2 Active Memory

To enable experience-driven learning within resource-constrained robotic systems, it is crucial to build a memory as a diverse set of demonstrations while minimizing human efforts and additional resources. To this end, we construct an active memory, M_A , consisting of demonstrations generated by an LLM, preserving its planning-oriented language and reasoning patterns. Notably, we retain both successful and failed demonstrations in M_A , as each provides complementary guidance in learning to capture the reasoning patterns behind success or failure.

The active memory is progressively updated by a selector module, $\mathcal{S}$, which employs a diversity-aware selection strategy inspired by active learning [59]. It evaluates both newly generated and stored demonstrations to identify the most informative and representative subset within a constrained memory budget. Given the current memory, M_A , and a set of newly generated demonstrations, G , containing successful and failed cases, the selector evaluates them in the language feature space to update the memory by discarding less informative or redundant examples and incorporating those that enhance diversity and representativeness in memory. This can be formalized as follows:

$$M_A \leftarrow \mathcal{S}(M_A, G), \text{ where } M_A = \{M_A^s, M_A^f\}. \quad (3.1)$$

Here, M_A^s and M_A^f represent the sets of selected successful and failed demonstrations, respectively.

To identify the most valuable demonstrations, we first encode them into dense vector representations using Sentence-BERT [60], capturing their semantic and contextual features. Then, we perform k-means clustering [61] on successful and failed demonstrations separately, aiming to ensure diversity within each case. The total number of clusters is set to the memory budget B , and the allocation between success and failure clusters can be adjusted, whose impact is analyzed in Section 3.3.1. From each cluster, we select a representative demonstration to retain in the active memory, keeping it compact yet informative. This reflects a wide range of planning behaviors and task-relevant knowledge.

3.2.3 Contrastive In-Context Learning with Dual Prompting

We introduce contrastive in-context learning with dual prompting to enhance in-context learning while adaptively leveraging two types of demonstrations. The key intuition is that effective learning benefits not only from understanding correct reasoning patterns but also from learning to recognize and avoid unsuccessful ones. This is motivated by the observation that LLMs are highly sensitive to the context and can imitate behaviors depending on the demonstrations they are given [62]. It can address the limitation of conventional prompting strategies, which primarily focus on positive examples [6], overlooking valuable learning signals from failure cases.

Accordingly, the active memory with successful demonstrations M_A^s and failure ones M_A^f is utilized for positive and negative prompting, respectively.

Positive prompting provides successful exemplars that demonstrate how to solve a task [6], whereas negative prompting introduces failure cases to help the model recognize and avoid incorrect strategies, inspired by [63]. Each demonstration $d_i = (x_i, y_i)$ is reconstructed into a triplet (g, x_i, y_i) , where g is an instruction signal determined by the prompting type as follows:

$$(g, x_i, y_i) = \begin{cases} (g_p, x_i, y_i), & \text{if } (x_i, y_i) \in M_A^s, \\ (g_n, x_i, y_i), & \text{if } (x_i, y_i) \in M_A^f. \end{cases} \quad (3.2)$$

Here, g_p is a positive instruction that encourages the model to imitate successful behavior, while g_n is a negative instruction that discourages the replication of failure. These instructions implicitly label the demonstrations, enabling dual prompting to contrast successes and failures within the context. Examples of positive and negative prompting are shown in Figure 3.3.

3.3 Experiments

3.3.1 Benchmark Experiments

Setup. *Datasets.* We validated the proposed method in three widely-used benchmark scenarios, including Shelf [57], Coffee [7], and Unpacking [58]. Each scenario is designed to validate the capability of the LLM-based task planning in multi-object, long-horizon, and spatially constrained manipulation tasks, respectively. In the Shelf scenario, the robot aims to re-arrange N objects, initially placed at random, into a size-ordered sequence on a shelf

that can hold up to $N + 1$ objects. The objective of the Coffee scenario is to serve N cups of coffee, each composed of up to six different ingredients, using one coffee machine. The Unpacking scenario requires retrieving a designated target object from a box filled with N objects of varying sizes and placing it into a separate box. Each environment was evaluated using 20 randomized test cases, designed with different initial and goal configurations. To evaluate performance across tasks of varying complexities, we adjusted the number of objects N in each scenario. Specifically, we set N to 2 and 4 for Coffee, and 5 and 7 for both Shelf and Unpacking. This variation produces task sequences of different lengths. In the Shelf scenario, the average sequence spans 15 and 26 steps for $N = 5$ and $N = 7$, respectively. In the Coffee scenario, the averages increase to 29 and 57 for $N = 2$ and $N = 4$, respectively. The Unpacking scenario has 5 and 7 steps for $N = 5$ and $N = 7$ ¹.

Implementation details. To verify the applicability of the proposed method across diverse models, we used two models: GPT-4o-mini (GPT) [64] and Gemini-1.5-Flash-8B (Gemini) [65] both of which have approximately 8 billion parameters. We utilized the LLM to convert natural language task instructions into PDDL representations [3], with a temperature of 0 to eliminate randomness in the output. After that, the LLM generated action sequences based on the translated PDDL, using a temperature of 0.1 to allow for diversity in plan generation. The success or failure of the generated plan

¹The task sequences are relatively short since accessing the target involves removing only a few obstructing objects in the box, not all items.

was assessed using a rule-based simulation system [17]. Additionally, we set the memory budget (B) to 6 and 12 demonstrations, updating the active memory every five task instructions to balance learning responsiveness and stability. Note that using 6 demonstrations is a common setting in LLM-based task planning [9, 13, 17] to manage token and inference costs of LLMs. To balance the ratios of successful and failed examples in memory, we limited the number of failed examples to at most half of the memory budget (see Table 3.3 for analyses of the failure ratio).

Comparison methods. We compared the proposed method against four representative baselines. (i) *Standard planning* performs in-context learning for task planning by using a fixed set of successful demonstrations with positive prompting [1–3]. (ii) *Re-planning* corrects errors of the generated action sequence through positive prompting with resolved correction examples as a recent self-corrective method [17]. (iii) *Planning with external success memory* [14] stores demonstrations that are either initially successful or become successful after re-planning and continues positive prompting with them. (iv) *Planning with external failure memory* [15, 16], stores the original failed demonstrations and their improved ones obtained through re-planning, along with feedback. For (iii) and (iv), we apply a recent self-corrective approach [17] for re-planning and use the widely adopted Sentence-BERT [60] to retrieve demonstrations with the closest embeddings to the task instruction. Note that the main difference between external memory of (iii) and (iv) and the proposed contrastive active memory is the memory capacity and

learning strategy. Specifically, the former external memory approaches expand their capacity by adding new demonstrations to positively learn LLM, but ours updates content within a fixed memory capacity to contrastively learn an LLM.

Results. *Main results.* We summarize the comparison results on three benchmark scenarios in Table 3.1, averaging over two memory budgets. The proposed method consistently outperforms all competitors, achieving an average increase of 10.5% with GPT and 11.9% with Gemini, respectively. Compared to Standard planning, our approach demonstrates a 18.8% higher average success rate across different models and scenarios. This indicates that relying on a pre-defined contextual memory limits the learning capability of the model and its adaptability in diverse scenarios. Although Re-planning requires additional resources, such as querying an LLM for plan validation and generating feedback, it shows lower average performance than our method with a 12.1% drop in GPT and a 14.2% drop in Gemini. Utilizing an external memory with successful examples yields comparable results to ours in some scenarios, but it shows an average performance drop of about 8.8% and 11.3% in the most challenging Unpacking scenario with GPT and Gemini, respectively. We observe that planning with an external failure memory consisting of corrected failures achieves performance similar to using successful examples, but it still results in an 5.0% and 7.9% average drop compared to our proposal in GPT and Gemini, respectively. Compar-

ing external success memory with standard planning and external failure memory with re-planning indicates that both approaches surpass their fixed memory counterparts with an average gain of 14.2% and 6.7%, respectively. It highlights the effectiveness of incorporating new demonstrations over relying on static context, though these improvements are marginal because they depend on only positive prompting. Those results suggest that incorporating both successful and failed demonstrations enhances the learning capabilities of LLMs by explicitly indicating which plans to imitate and which to avoid, thereby compensating for the limited guidance implicitly offered in successful examples.

Table 3.1: Comparison results of task success rates. The bold and underlined texts indicate the best and second-best results, respectively.

Method	Shelf		Coffee		Unpacking		Avg.
	$N = 5$	$N = 7$	$N = 2$	$N = 4$	$N = 5$	$N = 7$	
GPT-4o-mini							
Standard planning	72.5%	35.0%	77.5%	62.5%	62.5%	32.5%	57.1%
Re-planning	90.0%	37.5%	85.0%	77.5%	67.5%	37.5%	65.8%
Planning w/ external success memory	95.0%	50.0%	90.0%	85.0%	77.5%	45.0%	73.8%
Planning w/ external failure memory	90.0%	47.5%	87.5%	82.5%	77.5%	52.5%	72.9%
Planning w/ active memory (ours)	97.5%	52.5%	90.0%	87.5%	82.5%	57.5%	77.9%
Gemini-1.5-Flash-8B							
Standard planning	52.5%	20.0%	70.0%	57.5%	35.0%	25.0%	43.3%
Re-planning	55.0%	20.0%	85.0%	55.0%	37.5%	22.5%	45.8%
Planning w/ external success memory	57.5%	30.0%	90.0%	80.0%	47.5%	25.0%	55.0%
Planning w/ external failure memory	65.0%	22.5%	85.0%	67.5%	52.5%	20.0%	52.1%
Planning w/ active memory (ours)	67.5%	27.5%	90.0%	80.0%	60.0%	35.0%	60.0%

Ablation study. We conducted an ablation study to investigate the impact of the proposed method including the active memory, dual prompting, and the selector. Active memory allows memory updates, otherwise fixed memory is used. Selector performs diversity-based updates for active memory, but without it, random updates are used. Our dual prompting uses both success and failure examples, whereas without it only success examples are used. The results are summarized in Table 3.2, averaged across two budgets for each scenarios. Employing only the active memory (second rows) yields a modest 9.6% and 8.0% improvement over not employing it (first rows) in GPT and Gemini, respectively. However, it still shows an average 11.2% drop compared to our proposed method across all scenarios. Interestingly, comparing the impact of the selector and dual prompting, we observe that the absence of each resulted in an average drop of 5.2% (fourth rows) and 6.7% (third rows), respectively, across all experiment settings. It indicates that dual prompting, which leverages successful and failed knowledge for contrastive in-context learning, improves the overall performance and adaptability of the model.

Table 3.2: Ablation study in terms of active memory, dual prompting, and selector.

Functionality			Shelf		Coffee		Unpacking		Avg.
Active memory	Dual prompting	Selector	$N = 5$	$N = 7$	$N = 2$	$N = 4$	$N = 5$	$N = 7$	
GPT-4o-mini									
			72.5%	35.0%	77.5%	62.5%	62.5%	32.5%	57.1%
✓			82.5%	40.0%	85.0%	75.0%	72.5%	45.0%	66.7%
✓		✓	92.5%	47.5%	87.5%	80.0%	72.5%	50.0%	71.7%
✓	✓		90.0%	45.0%	87.5%	82.5%	75.0%	50.0%	71.7%
✓	✓	✓	97.5%	52.5%	90.0%	87.5%	82.5%	57.5%	77.9%
Gemini-1.5-Flash-8B									
			52.5%	20.0%	70.0%	57.5	35.0%	25.0%	43.3%
✓			60.0%	22.5%	80.0%	72.5%	45.0%	27.5%	51.3%
✓		✓	62.5%	25.0%	80.0%	75.0%	47.5%	27.5%	52.9%
✓	✓		65.0%	25.0%	85.0%	75.0%	52.5%	32.5%	55.8%
✓	✓	✓	67.5%	27.5%	90.0%	80.0%	60.0%	35.0%	60.0%

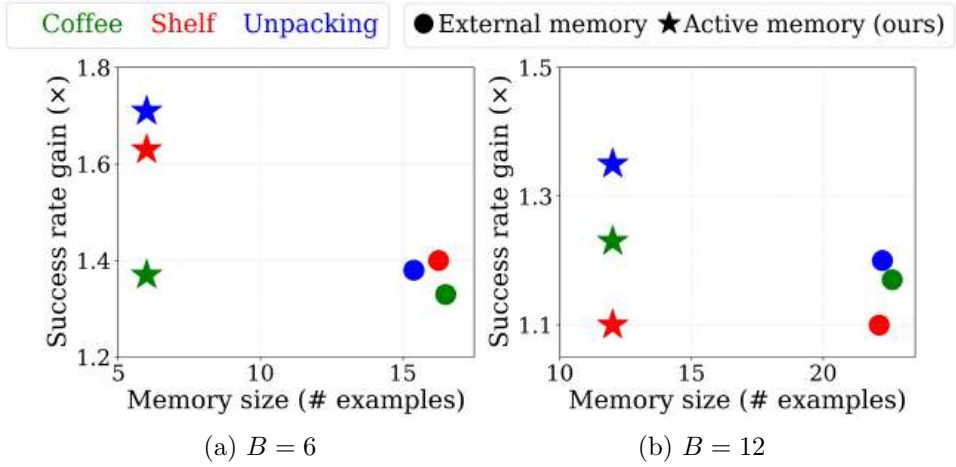


Figure 3.4: Comparison of task success rate and memory size. Success rate gain denotes the success rate improvement compared to Standard planning. Best viewed in color.

Efficiency of active memory. We evaluated our proposed active memory and the external memory approach [14–16], comparing their average success rate gain over standard planning and memory size, as shown in Figure 3.4. External memory methods expand as new demonstrations accumulate over time, thereby requiring larger memory resources, averaging about 2.67 times and 1.86 times the size of the proposed active memory with 6 and 12 budgets, respectively. They achieve a 1.26 success rate gain over standard planning across all scenarios, whereas our proposal achieves an higher average gain of 1.38, demonstrating superior performance with substantially less memory. These findings indicate that integrating newly generated knowledge while optimizing memory allocation can cut memory usage without compromising performance.

Impact of prompt ratio. We analyzed the impact of varying the ratio of

Table 3.3: Analysis of prompt ratio. The prompt ratio denotes the maximum proportion of negative prompts within the active memory. The reported values indicate task success rate gains over the 0% prompt ratio baseline.

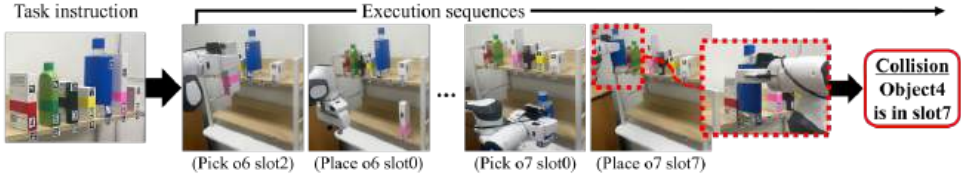
Prompt ratio	0%	33%	50%	67%
<i>B</i> = 6				
Easy	-	+4.2%	+11.7%	<u>+5.8%</u>
Hard	-	<u>+5.0%</u>	+8.3%	+1.7%
<i>B</i> = 12				
Easy	-	+0.8%	+4.2%	<u>+1.7%</u>
Hard	-	<u>+2.5%</u>	+3.3%	+1.7%
Avg.				
Easy	-	+2.5%	+7.9%	<u>+3.8%</u>
Hard	-	<u>+3.8%</u>	+5.8%	+1.7%

negative prompts given an active memory budget (B) of 6 and 12, as shown in Table 3.3. We evaluated four configurations using GPT and Gemini: 0% with positive prompts only and allowing up to 33%, 50%, and 67% negative prompts out of each memory budgets, respectively. For the analysis, we grouped scenarios into two categories, with settings for each scenario with fewer objects considered as Easy (i.e., Shelf and Unpacking for $N = 5$, and Coffee for $N = 2$) and settings with more objects considered as Hard (i.e., the other cases with larger N in each scenario). Then, we calculated the average success rate for each category and reported the improvements over the method using only positive prompts (0%). On average, the results show that maintaining a balanced ratio of positive and negative signals yields the best performance. Notably, increasing the proportion of negative prompts tends to benefit Easy, whereas increasing the proportion of positive prompts is more effective for Hard.

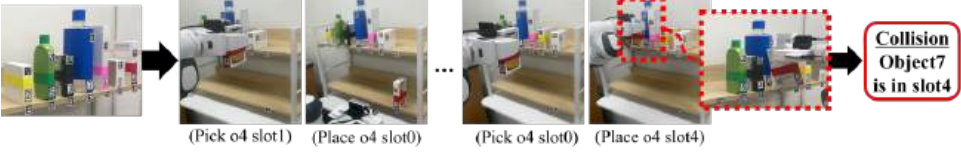
3.3.2 Real-World Experiments

Setup. We evaluated the proposed method using a 7-DOF Franka Emika Panda robot arm in the Shelf scenario using AprilTag [66] for scene perception with a wrist camera. The robot rearranges seven objects of different sizes on a shelf according to a specified size-based order using Gemini with $B = 6$. Note that the shelf has one extra slot available in the first floor. To gather scene information for all objects and slots in the first and second floors of the shelf, the robot returns to detecting positions after each action. We utilized the Moveit [67] for the motion planning, which lies beyond the scope of this section. To verify the effectiveness of our method, we compared the generated action sequences against the Standard planning approach. This allows us to evaluate both the planning performance and the impact of the proposed contrastive in-context learning with successes and failures.

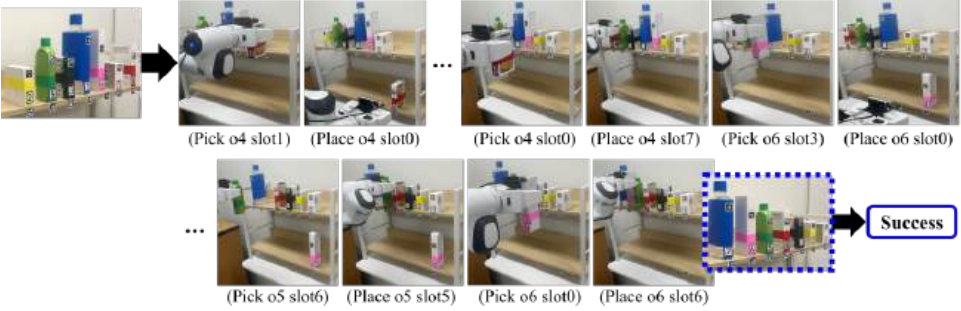
Results. We visualized the actual execution sequence performed by the robot as in Figure 3.5. Note that in our notation, `(action oi slotj)` represents performing an action on object i (denoted by `oi`), with respect to `slotj`. As shown in Figure 3.5(a), both the standard and proposed planning approaches fail when executing `(place o7 slot7)`, since `slot7` is already occupied by `object4`, resulting in a collision. Then, the standard planning method fails again given a similar task instruction, as illustrated in Figure 3.5(b). Since it does not leverage the previous failed experience for learning, resulting in a similar collision when attempting `(place o4 slot4)`,



(a) Failed demonstration



(b) Standard LLM-based planning



(c) Planning with contrastive in-context learning (proposed method)

Figure 3.5: Comparison of execution sequences in the Shelf scenario between the standard LLM-based planning and the proposed planning method. (action oi slot j) represents that the action targets object i (denoted by oi), taking slot j into account, where $i = 1, \dots, 7$, and $j = 0, \dots, 7$. Best viewed in color ($\times 2$).

where slot4 is already occupied by object7. However, our method leverages the previous failed experience through contrastive in-context learning, allowing the model to learn to avoid similar mistakes and complete a similar task in the future. Therefore, our proposal successfully completes the task goal by generating a plan that avoids the similar collision in experienced failures, as shown in Figure 3.5(c).

3.4 Discussion

In this chapter, we have introduced active knowledge acquisition framework through a contrastive in-context learning for task planning with large language models. It employs a dual prompting strategy, where successful knowledge serves as positive prompts to guide the model toward effective behaviors, and failed knowledge is used as negative prompts to help avoid previously observed mistakes. It allows the model to learn from the contrast between the two different prompts. To leverage both types of knowledge for learning, we introduce an active memory that is progressively updated with task-specific demonstrations, generated by LLMs during task planning, ensuring the retention of a compact but diverse set of knowledge. Extensive evaluations in both benchmark environments and real-world settings shows that our method achieves a markedly higher task completion rate than existing LLM-based planners while requiring only minimal additional resources.

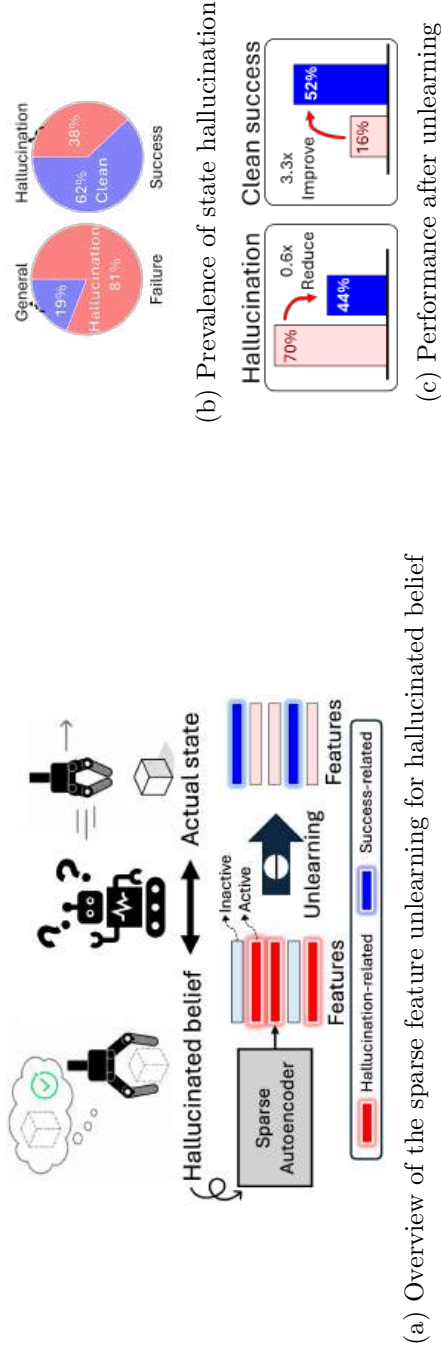
4. Selective Knowledge Removal through Policy Unlearning

Vision-language-action models (VLAs) have become a promising approach for bringing foundation models into robot manipulation [27–29]. By integrating language-conditioned task reasoning with visual grounding and low-level action generation, these models enable translation of high-level instructions into continuous robot control. Their success suggests that robot policies can benefit from multi-modal pretrained representations and large-scale heterogeneous robot data, enabling broader generalization across tasks, objects, and embodiments [30, 31, 37]. However, as VLAs move toward real-world deployment, their reliability becomes a central concern [52, 68, 69]. Unlike hallucination in text or vision-language generation, hallucinated actions in VLAs directly affect physical execution and can lead to infeasible or unsafe interactions with the environment [69].

In this work, we investigate state hallucination, a critical failure mode of VLAs in which the policy generates actions based on an incorrect implicit belief about the robot-object state. For instance, a VLA may close its gripper before reaching the object and then continue transport and place motions as if the grasp had succeeded, producing a sequence of actions incon-

sistent with the actual scene (Figure 4.2). Our analysis on OpenVLA [28] with LIBERO-Plus [68] shows that state hallucination is *prevalent*, occurring in 70% of rollouts and being harmful in 60%. Such hallucination rollouts show weakened attention on task-relevant regions such as the gripper and target object and more scattered attention over the background. Through a mechanistic analysis with sparse autoencoders (SAEs), we further identify hallucination-relevant sparse features that emerge and activate when these failures occur. These findings establish state hallucination as a diagnosable failure mode of VLAs.

Based on this analysis, we propose *Region-Guided Feature Policy Unlearning*, which selects sparse features that are strongly associated with hallucination in specific visual regions and suppresses their activations while preserving normal behaviors. Specifically, we identify region-feature pairs whose activation correlates with hallucination failures, and optimize a feature-level unlearning objective that suppresses these features without erasing the general manipulation capability of the VLA. Experiments on LIBERO-Plus and a real-world Franka robot show that our method substantially reduces harmful hallucination and improves task success while preserving the original manipulation capability, suggesting that mechanistic feature analysis provides a practical basis for selectively unlearning undesirable behaviors in embodied foundation models.



(a) Overview of the sparse feature unlearning for hallucinated belief

Figure 4.1: Sparse feature unlearning mitigates hallucinated beliefs in VLAs. (a) State hallucination occurs when VLA mistakes its hallucinated belief for the actual robot-object state and generates actions based on this false belief. Hallucination-related features are activated during this mismatch, which are then suppressed through the proposed unlearning while preserving success-related features. (b) Hallucination is a dominant failure mode, accounting for 81% of failures, and remains frequent even in successful trials at 38%. (c) Our proposed sparse feature unlearning reduces hallucination from 70% to 44% and improves clean success without hallucination from 16% to 52%, showing that targeted feature suppression can mitigate state hallucination while improving task completion.

4.1 Related Works

4.1.1 Machine Unlearning

Machine unlearning aims to make a trained model forget the knowledge associated with selected data without retraining the model from scratch [70–72]. Prior work has studied this problem mainly in classification tasks, where selected examples are removed, and in language-model tasks, where factual knowledge or undesired responses are suppressed [73,74]. A central challenge in machine unlearning is the trade-off between removing undesired knowledge and preserving useful model capabilities. This trade-off is especially important for robot action policies, where undesired behaviors can share perception, language grounding, and motor-control representations with normal skills. This motivates selective policy unlearning methods that suppress representations associated with harmful behaviors while preserving representations needed for successful task execution. In this work, we study policy unlearning for VLAs by selectively suppressing internal features associated with state hallucination while preserving clean task-execution capability.

4.1.2 Sparse Autoencoders

Sparse autoencoders (SAEs) provide a way to analyze high-dimensional model activations by decomposing dense hidden representations into sparse feature activations [75,76]. Each sparse feature is activated only for a subset of inputs, making it easier to associate internal model activity with specific

inputs or behaviors. Top- k SAEs are widely used because they enforce sparsity with a simple and explicit constraint: only the k largest latent activations are retained and the remaining activations are set to zero for each input [76, 77]. In this work, we employ SAEs to analyze and mitigate state hallucination in VLAs. Specifically, we identify hallucination-associated features across visual regions, and suppress the corresponding feature activations through region-guided policy unlearning.

4.2 State Hallucination in Vision Language Action Models

In this section, we show that state hallucination is a primary cause of task failures in current VLAs and analyze where it manifests internally. We first define state hallucination (§4.2.1), then quantify its prevalence (§4.2.2), compare attention patterns between normal and hallucination executions (§4.2.3), and finally probe the internal features that trigger it through sparse autoencoders (§4.2.4).

4.2.1 Definition

We define *state hallucination* as the phenomenon in which a VLA model generates actions that presuppose a robot-object state different from the actual one. Figure 4.2 illustrates this phenomenon with two representative cases, each comparing a normal and a hallucination execution under the same

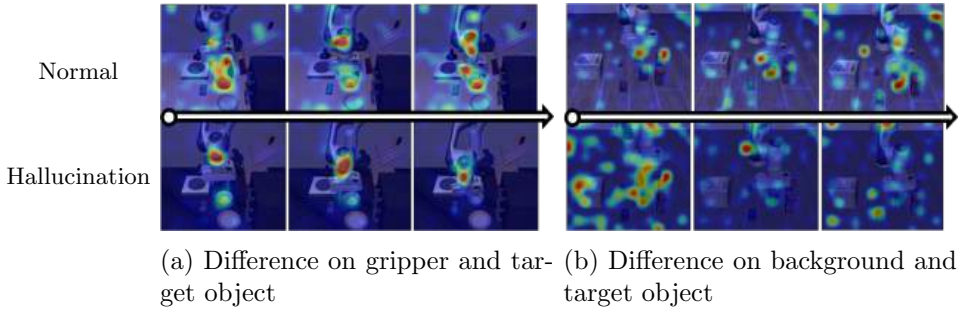


Figure 4.2: What is state hallucination? State hallucination refers to the phenomenon in which a VLA generates actions based on a predicted robot-object state that differs from the actual one, leading to actions inconsistent with the true state. We illustrate it with two representative cases comparing normal (top) and hallucination (bottom) executions under the same task instruction. In both cases, state hallucination coincides with a visible breakdown of attention on task-relevant regions.

task instruction. In the left figure, normal execution maintains strong attention on the gripper and target object before the grasp event. In contrast, the hallucination execution shows weaker attention on these task-relevant regions. In the right figure, the model executes a misdirected action toward a non-target region; its attention is scattered over the background and non-target objects, in contrast to the normal execution that concentrates on the gripper and the target object. Across both cases, state hallucination coincides with a visible breakdown in attention to task-relevant regions, suggesting that attention failure is a behavioral pattern of state hallucination.

4.2.2 Characterization

To quantify the significance of state hallucination in task failures, we analyze the rollouts of OpenVLA [28] on LIBERO-Plus [68] and measure how

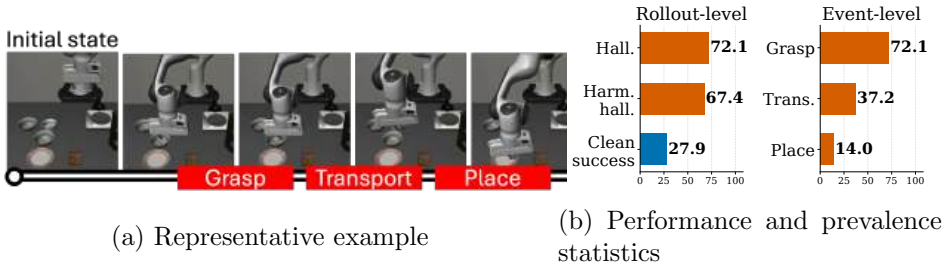


Figure 4.3: How does state hallucination appear in VLA execution? (a) The robot closes the gripper while approaching, transports as though it were securely grasped, and places even though the object is not actually held. (b) Performance and prevalence statistics show frequent state hallucination, with grasp being the most prevalent type. ‘Hall.’ and ‘Trans.’ denote state hallucination and transport, respectively.

frequently state hallucination occurs in Figure 4.3. Figure 4.3a illustrates the progress of states during robot manipulation. As shown in Figure 4.3b, state hallucination appears in 70% of all rollouts, and 60% are harmful hallucinations that directly result in task failure, while only 26% achieve clean success. To further characterize when hallucination occurs, we categorize harmful hallucinations by the manipulation events, which grasp hallucination is the dominant mode, followed by transport and place. These results confirm that state hallucination is not a rare failure mode but a dominant cause of task failure in current VLAs.

4.2.3 Attention Analysis

We quantify attention from the normalized maps using two metrics. First, we measure region-level attention by averaging the normalized attention values over the visual tokens assigned to each semantic region. Sec-

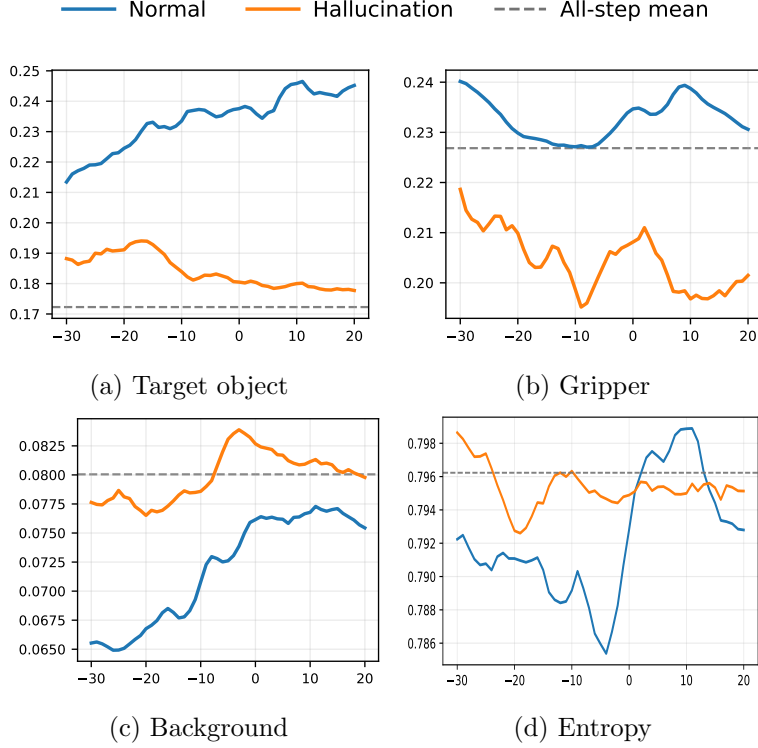


Figure 4.4: Quantitative comparison of event-aligned attention patterns between normal and hallucination executions. The gray dashed line indicates the mean values across all steps. The x-axis is aligned to the corresponding event, where $x = 0$ denotes the time step at which the event occurs.

ond, we measure attention entropy to quantify how spatially dispersed the attention is over visual tokens. As plotted in Figure 4.4, normal executions assign higher attention to task-relevant regions, especially the target object and gripper. In contrast, hallucination executions assign lower attention on these regions, while assigning relatively higher attention to the background. In terms of attention entropy, normal executions show lower than mean value before the event, indicating more concentrated attention. In contrast, hallucination executions remain near the mean entropy level, suggesting weaker

event-related attention concentration.

4.2.4 Mechanistic Analysis

To investigate the internal mechanisms underlying state hallucination, we scope the triggering robot representations. Specifically, we decompose the representations of VLA into sparse and interpretable representations using Sparse Autoencoder (SAE) [75], which allows us to identify the specific features associated with hallucination behavior.

SAE training details. We use Top- k SAEs [76,77] to encode activations into sparse feature representations. To train these SAEs, we construct an activation dataset from rollouts under three outcome conditions: clean success (CS), hallucination failure (HF), and non-hallucination failure (NF), allowing us to distinguish hallucination-specific features from those related to clean execution or general failure. Each SAE learns to reconstruct an activation from a sparse representation, where only the k largest latent activations are retained and the remaining values are set to zero. After training, we freeze the SAE and use its encoder to map policy activations into sparse feature activations.

Hallucination score derivation. Motivated by the region-level attention differences observed in Section 4.2.3, we condition SAE feature activations on semantic visual regions. For an input sample x , let $\mathcal{R}_r(x)$ denote the set of visual tokens belonging to region r . Let $z_{p,j}(x)$ denote the activation of

SAE feature j at visual token p . We define the region-conditioned feature activation as

$$z_{r,j}(x) = \frac{1}{|\mathcal{R}_r(x)|} \sum_{p \in \mathcal{R}_r(x)} z_{p,j}(x). \quad (4.1)$$

This measures how strongly feature j is expressed in semantic region r .

We next quantify how each region-conditioned feature is expressed across execution outcomes. Since hallucination-related features may be activated only within short event windows, we summarize each rollout by the maximum activation of each region-feature pair over time, $m_{e,r,j} = \max_t z_{e,t,r,j}$. We then average this rollout-level activation within each execution outcome:

$$\bar{m}_{c,r,j} = \frac{1}{|\mathcal{E}_c|} \sum_{e \in \mathcal{E}_c} m_{e,r,j}, \quad c \in \{\text{CS}, \text{NF}, \text{HF}\}. \quad (4.2)$$

These group-wise activations quantify how strongly each region-feature pair is expressed across the three groups. For each pair (r, j) , we define two complementary scores, namely a hallucination-specific score $s_{r,j}^{\text{hall}}$ and a clean-specific score $s_{r,j}^{\text{clean}}$:

$$s_{r,j}^{\text{hall}} = \bar{m}_{\text{HF},r,j} - \frac{1}{2} (\bar{m}_{\text{CS},r,j} + \bar{m}_{\text{NF},r,j}), \quad s_{r,j}^{\text{clean}} = \bar{m}_{\text{CS},r,j} - \frac{1}{2} (\bar{m}_{\text{HF},r,j} + \bar{m}_{\text{NF},r,j}). \quad (4.3)$$

Here, a larger $s_{r,j}^{\text{hall}}$ indicates that feature j at region r is selectively activated in hallucination failures compared with the other groups. Conversely, a larger $s_{r,j}^{\text{clean}}$ indicates stronger specificity to clean successful executions. We select the top-ranked features by $s_{r,j}^{\text{hall}}$ and $s_{r,j}^{\text{clean}}$ as hallucination-relevant

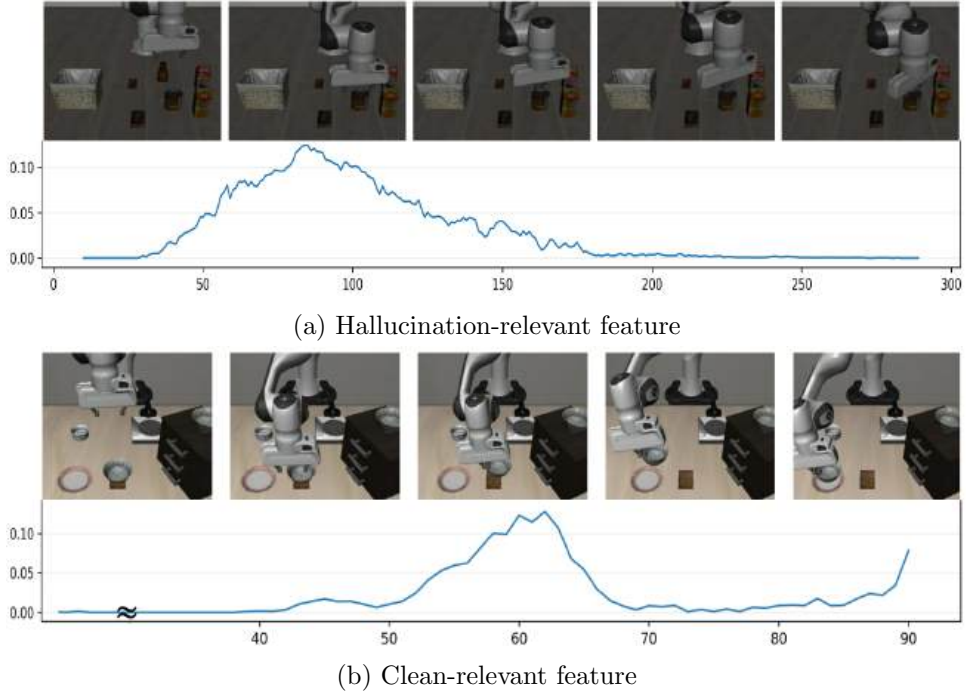


Figure 4.5: Sparse features distinguish hallucination and clean executions. We visualize representative sparse feature activations over rollout trajectories. (a) A hallucination-relevant feature is strongly activated when grasp hallucination occurs. (b) A clean-relevant feature is activated when the policy successfully executes the corresponding subtask and completes the overall task.

and clean-relevant features, respectively.

Sparse feature analysis. Figure 4.5 illustrates how these selected features appear in rollouts: the hallucination-relevant feature becomes strongly activated around a grasp hallucination, whereas the clean-relevant feature is activated during successful subtask execution and task completion. These results indicate that sparse features capture behaviorally meaningful signals associated with hallucination and clean execution.

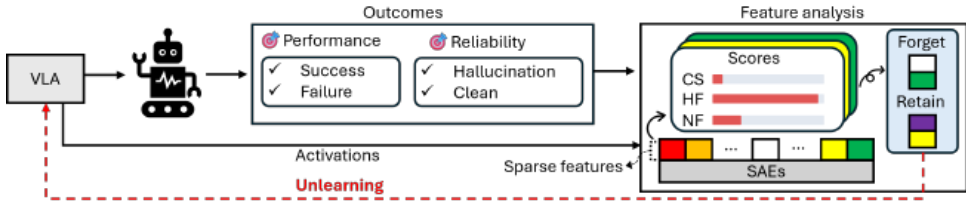


Figure 4.6: Overall framework of the proposed method. First, we identify visual-region-specific sparse features associated with hallucination and clean rollouts from VLA activations. Second, we unlearn the policy with a feature-level unlearning objective that suppresses the selected hallucination-associated features while preserving clean-associated features needed for normal behaviors.

4.3 Proposed Method

Our goal is to reduce state hallucination while preserving normal behaviors. The analysis in Section 4.2 suggests that hallucination is associated with changes in visual grounding over semantic regions and sparse internal features. Based on this analysis, we propose *Region-Guided Feature Policy Unlearning*, which selectively suppresses hallucination-associated sparse features grounded in specific visual regions, as illustrated in Figure 4.6.

4.3.1 Region-Guided Feature Selection

We construct feature sets for forgetting and retention based on the hallucination-specific and clean-specific scores defined in Section 4.2.4. To further isolate features specific to hallucination failures, we introduce a separation score as $s_{r,j}^{\text{sep}} = \bar{m}_{\text{HF},r,j} - \bar{m}_{\text{NF},r,j}$, which measures specificity to hallucination failures over general failures. For the forget set $\mathcal{F}$, we rank region-feature pairs using an aggregated score $s_{r,j}^{\mathcal{F}} = s_{r,j}^{\text{hall}} + s_{r,j}^{\text{sep}} - s_{r,j}^{\text{clean}}$. This score pri-

oritizes features that are active in hallucination failures, distinct from non-hallucination failures, and weakly associated with clean execution. We select the top-ranked pairs as the forget set $\mathcal{F}$. For the retain set $\mathcal{G}$, we select region-feature pairs with high $s_{r,j}^{\text{clean}}$. The resulting sets guide unlearning to suppress state-hallucination-associated features while preserving features needed for clean manipulation.

4.3.2 Feature-Guided Policy Unlearning

After selecting the forget set $\mathcal{F}$ and retain set $\mathcal{G}$, we update the VLA to suppress hallucination-associated features while preserving clean-associated features. During unlearning, the SAE encoders are frozen, and only the VLA parameters are updated with LoRA [78].

For hallucination samples, the forget loss penalizes the selected forget pairs, while for clean successful samples, the retain loss encourages the selected retain pairs to remain active:

$$\begin{aligned}\mathcal{L}_{\text{forget}} &= \mathbb{E}_{x \in \mathcal{D}_{\text{HF}}} \left[\frac{1}{|\mathcal{F}|} \sum_{(r,j) \in \mathcal{F}} (z_{r,j}(x; \theta))^2 \right], \\ \mathcal{L}_{\text{retain}} &= -\mathbb{E}_{x \in \mathcal{D}_{\text{CS}}} \left[\frac{1}{|\mathcal{G}|} \sum_{(r,j) \in \mathcal{G}} z_{r,j}(x; \theta) \right].\end{aligned}\tag{4.4}$$

Combining the forgetting and retention terms, the final unlearning objective is defined as $\mathcal{L}_{\text{unlearn}} = \mathcal{L}_{\text{forget}} + \lambda_r \mathcal{L}_{\text{retain}}$, where λ_r controls the strength of retention.

4.4 Experiments

4.4.1 Benchmark Experiments

Setup. We conduct our analysis and evaluation on LIBERO-Plus [68] using OpenVLA [28], a widely used and representative autoregressive model. LIBERO-Plus extends LIBERO [79] with realistic visual perturbations and multiple severity levels, making it suitable for evaluating execution reliability. To provide a reliable reference for comparing normal and hallucination executions, we select tasks on which the original policy achieves stable success in the corresponding original LIBERO. Specifically, we use representative task instructions from the LIBERO-Plus suites: *put the bowl on the stove* from the goal suite, *pick up the black bowl on the cookie box and place it on the plate* from the spatial suite, and *pick up the tomato sauce and place it in the basket* from the object suite. More implementation details are provided in the supplementary materials.

Metrics. We report both task-level and hallucination-level metrics. *Overall success* measures the percentage of executions that complete the task. *Clean success* measures the percentage of executions that complete the task without false-belief hallucination. *Hallucination* measures the percentage of executions containing at least one false-belief hallucination event. *Harmful hallucination* measures the percentage of executions that contain false-belief hallucination and fail the task. We also report event-level hallucination rates for *grasp*, *transport*, and *place*. Lower hallucination and harmful hallucina-

tion are better, while higher overall success and clean success are better.

Results. *Does the proposed region-guided feature unlearning improve reliability?* To evaluate whether feature-level unlearning improves policy reliability, we compare three policies: the original pre-trained policy without unlearning, a naive gradient-ascent unlearning, and our proposed region-guided feature unlearning method. The gradient-ascent method directly increases the loss on hallucination samples, whereas our method suppresses hallucination-associated SAE features only in the semantic regions where they are selected, while retaining features associated with clean executions. As in Table 4.1, the original policy exhibits frequent action hallucination, with 70% hallucination and 60% harmful hallucination, while achieving 16% clean success. Naive gradient ascent does not provide meaningful mitigation. Instead, it increases hallucination to 84% and reduces both overall success and clean success to 0%. This suggests that directly pushing the model away from hallucination samples disrupts the underlying manipulation policy rather than selectively removing hallucination-related behavior. In contrast, our region-guided feature unlearning reduces hallucination to 44% and harmful hallucination to 38%, while improving overall success to 58% and clean success to 52%. These results indicate that the proposed method improves reliability by suppressing hallucination-related failures without erasing useful manipulation capabilities.

Table 4.1: Performance comparison in terms of hallucination and task success.

Method	Hallucination ↓	Harmful hallucination ↓	Overall success ↑	Clean success ↑	Grasp ↓	Transport ↓	Place ↓
Original Policy	70.0	60.0	26.0	16.0	70.0	38.0	16.0
Gradient Ascent	84.0	84.0	0.0	0.0	84.0	62.0	0.0
Region-Guided Feature	44.0	38.0	58.0	52.0	44.0	20.0	6.0

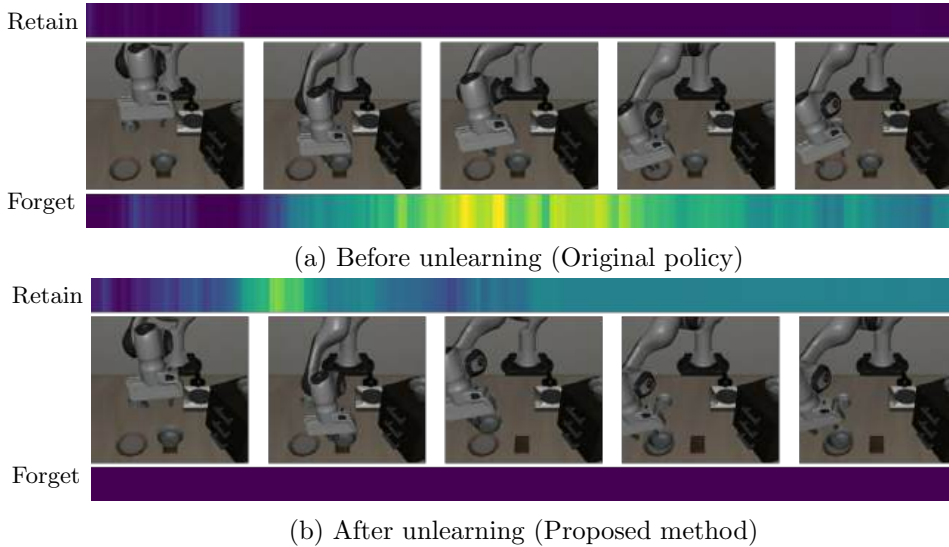


Figure 4.7: Comparison of rollouts and selected forget and retain feature activations before and after unlearning on the benchmark.

Do the selected features reflect hallucination- and clean-related behavior?

Figure 4.9 qualitatively compares the temporal activations of the selected retain and forget features before and after region-guided feature unlearning. Before unlearning, the original policy produces an action hallucination and fails to complete the task. During this hallucination behavior, the features selected as the forget set, which are associated with hallucination-related executions, are strongly activated, whereas the retain-set features, which are associated with clean successful executions, remain almost inactive. After applying our method, the policy successfully completes the task without hallucination. In this case, the hallucination-related forget features are largely suppressed, while the clean-related retain features become activated throughout the successful manipulation sequence. These results show that

Table 4.2: Comparison between global, tokenized, and region-guided feature unlearning. Changes are relative to the original policy.

Method	Hallucination ↓	Harmful hallucination ↓	Overall success ↑	Clean success ↑
Global Feature	62.0 (-8.0)	56.0 (-4.0)	34.0 (+8.0)	28.0 (+12.0)
Tokenized Feature	60.0 (-10.0)	50.0 (-10.0)	44.0 (+18.0)	34.0 (+18.0)
Region-Guided Feature	44.0 (-26.0)	38.0 (-22.0)	58.0 (+32.0)	52.0 (+36.0)

the selected forget and retain features correspond to hallucination-related and clean task-relevant behavior, respectively, and that the proposed unlearning objective suppresses hallucination-related activations while preserving clean task-relevant activations.

How effective is region-guided feature selection? To evaluate the effect of visual grounding in feature selection through SAEs, we compare three variants that use different levels of spatial information. *Global Feature* pools feature activations over all visual tokens. *Tokenized Feature* keeps feature activations at the visual-token level. *Region-Guided Feature*, our proposed variant, assigns visual tokens to semantic regions and selects features according to their region-conditioned activations. As in Table 4.2, *Global Feature* reduces hallucination but provides limited improvement in task success, suggesting that global pooling can lose spatial information. *Tokenized Feature* improves over the global variant, indicating that preserving token-level localization helps identify hallucination-associated features more effectively. Our *Region-Guided Feature* achieves the best overall trade-off, reducing the hallucination metrics by an average of 24.0% and improving the success metrics by an average of 34.0% over the perturbation baseline. These results suggest

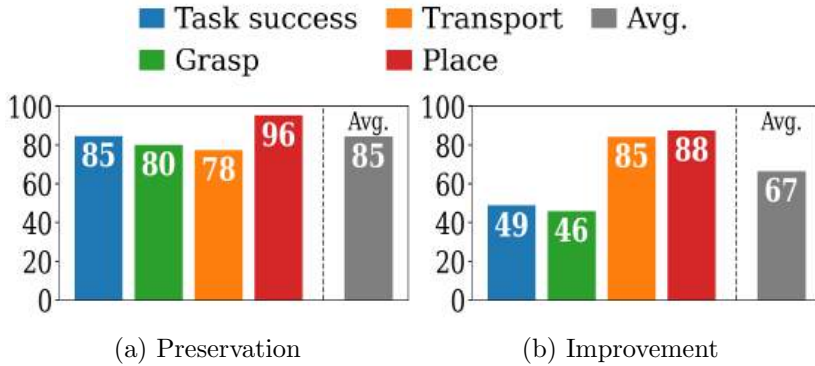


Figure 4.8: Unlearning trade-offs. Preservation measures retained correct behaviors, while improvement measures failures corrected after unlearning.

that region guidance enables more selective feature unlearning by focusing on hallucination-associated features within task-relevant regions while preserving features for successful manipulation.

Does the proposal preserve clean successful behaviors while improving hallucination failures? We evaluate the unlearning trade-off between retaining clean behaviors and removing hallucination-related behaviors. We measure this using preservation and improvement at both the task and event levels. At the task level, preservation indicates that originally successful executions remain successful, while improvement indicates that previously failed executions become successful. At the event level, preservation indicates that non-hallucinated events remain non-hallucinated, while improvement indicates that previously hallucinated events are suppressed. As shown in Figure 4.8, the proposed method preserves 85% of originally successful task executions after unlearning. At the event level, it preserves non-hallucinated grasp, transport, and place events at rates of 87%, 75%, and 91%, respec-

Table 4.3: Ablation study of score aggregation and the retain objective.

$s_{r,j}^{\text{forget}}$	$s_{r,j}^{\text{agg}}$	$\mathcal{L}_{\text{retain}}$	Hallucination ↓	Harmful hallucination ↓	Overall success ↑	Clean success ↑
✓	✓	✓	44.0 (-26.0)	38.0 (-22.0)	58.0 (+32.0)	52.0 (+36.0)
✓	✓		46.0 (-24.0)	36.0 (-24.0)	58.0 (+32.0)	48.0 (+32.0)
✓		✓	54.0 (-16.0)	50.0 (-10.0)	48.0 (+22.0)	44.0 (+28.0)

tively. The method also improves previously failed behavior, converting 49% of failed task executions into successful ones and suppressing 73% of event-level hallucinations on average across grasp, transport, and place. These results suggest a favorable unlearning trade-off, where hallucination-related failures are substantially reduced with limited degradation of clean behaviors.

Do score aggregation for feature selection and retention loss improve the reliability? Table 4.3 ablates the target-selection score and the retain objective used in our method. Note that the aggregated score $s_{r,j}^{\text{agg}}$ further uses $s_{r,j}^{\text{sep}}$ to filter out general failure features and $-s_{r,j}^{\text{retain}}$ to avoid suppressing clean-associated features. Using $s_{r,j}^{\text{agg}}$ without $\mathcal{L}_{\text{retain}}$ achieves strong hallucination reduction, but clean success decreases to 48%. Using $s_{r,j}^{\text{forget}}$ with $\mathcal{L}_{\text{retain}}$ preserves clean success better, but yields weaker hallucination reduction. Combining $s_{r,j}^{\text{agg}}$ with $\mathcal{L}_{\text{retain}}$ achieves the best overall trade-off, reducing hallucination and harmful hallucination to 44% and 38%, while improving overall success and clean success to 58% and 52%. These results show that score aggregation improves the selectivity of forget target selection, while the retain objective helps preserve clean manipulation behavior.

4.4.2 Real-World Experiments

Setup. We evaluate the proposed method in a real-world kitchen environment using a 7-DoF Franka Research 3 (FR3) robot, with the task instruction: “Pick fries and put it on the plate.”

Results. Does the proposed method remain effective in real-world environments? As shown in Figure 4.8, the original policy achieves only 24% clean success and exhibits a high harmful hallucination rate of 52%, indicating that task failures are frequently associated with hallucination-prone behavior rather than simple low-level execution errors. During these hallucinated executions, the forget features selected by our SAE-based analysis are strongly activated, whereas the retain features associated with clean behavior remain largely inactive. After applying the proposed feature unlearning method, the robot achieves a clean success rate of 36%, with a 12% reduction in harmful hallucination. Consistently, the hallucination-related forget features become suppressed, while the clean-related retain features are activated. These real-world results suggest that the proposed feature-level unlearning is not limited to simulated benchmarks, but can also suppress hallucination-related behaviors and improve policy reliability in physical robot execution.

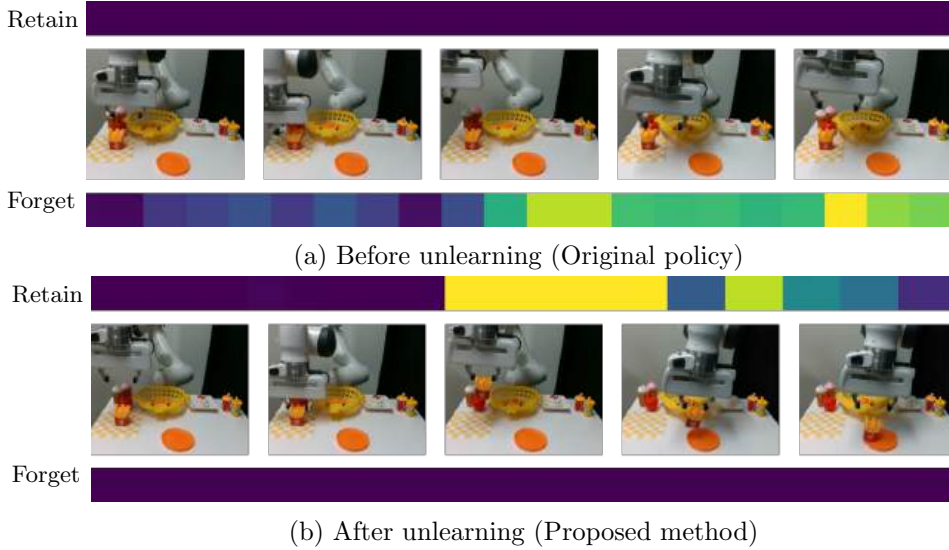


Figure 4.9: Comparison of rollouts and feature activations before and after unlearning on the real robot.

4.5 Discussion

In this chapter, we have presented a selective knowledge removal framework for vision-language-action models through policy unlearning. We focused on state hallucination in vision-language-action models, where a policy generates actions based on an incorrect implicit belief about the robot-object state. Our analysis showed that state hallucination is reflected in weakened attention to task-relevant visual regions and in hallucination-relevant sparse features identified by sparse autoencoders. Based on these findings, we proposed Region-Guided Feature Policy Unlearning, which selectively suppresses hallucination-associated region-feature pairs while retaining features needed for successful execution. Experiments in simulated and real-world environments show that our method reduces harmful hallucination

and improves task success without compromising the original manipulation capability. These results suggest that mechanistic feature analysis provides a practical basis for diagnosing and unlearning undesirable behaviors, offering a step toward more interpretable and reliable VLA-based robot manipulation policies.

5. Self-Corrective Knowledge Refinement through Feedback-Guided Inverse Reasoning

Recent advancements in large language models (LLMs) have demonstrated remarkable capabilities across various tasks, including translation [54], code generation [55], arithmetic [80], and commonsense reasoning [6]. Their ability to comprehend the context and generate complex outputs based on extensive knowledge about the world has led to growing attention in robot task planning, where precise decision-making and adaptability are crucial [1–4].

However, LLMs continue to produce outputs that, while plausibly worded, are often inaccurate or infeasible [81]. This issue extends to robot task planning, where the models often generate task plans that may appear syntactically sound but are non-executable by robots [82]. To address these planning errors from LLMs, existing works have been investigated through two approaches: external validator and self-correction methods (see Figure 5.1(b) and 5.1(c)). External validation methods are widely employed using pre-defined sets of errors [18, 83, 84] or external knowledge sources [7]. Some

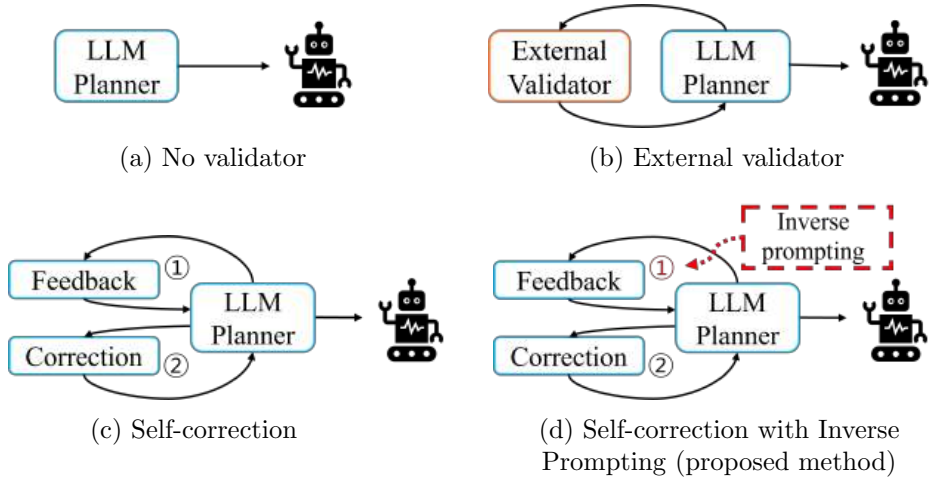


Figure 5.1: Given a goal, the LLM planner generates an action sequence. Then, (a) a robot executes it without any validation. (b) The generated action sequence is validated by an external validator, which is constructed using rule-based methods or by retrieving knowledge from external sources. (c) The LLM planner validates and refines the generated action sequence through a self-correction process, providing feedback on its output and using it to correct actions. (d) The proposed method further enhances the self-correction process from (c) with inverse prompting.

approaches construct predefined error types and corresponding feedback in advance to compare against the generated plan [18, 83, 84]. While effective for certain errors, these approaches are constrained by their reliance on fixed rules, which may not address all possible errors or adapt to new scenarios. They also require substantial human effort to construct strict rules. Another line of work retrieves knowledge from external sources, such as news articles, Wikipedia [85, 86], or additional LLMs for verification [7]. However, accessing these resources may not always be feasible due to high costs [87].

To overcome the reliance on additional resources, a self-correction method [88] has been applied to robot task planning [17, 89]. Their approaches en-

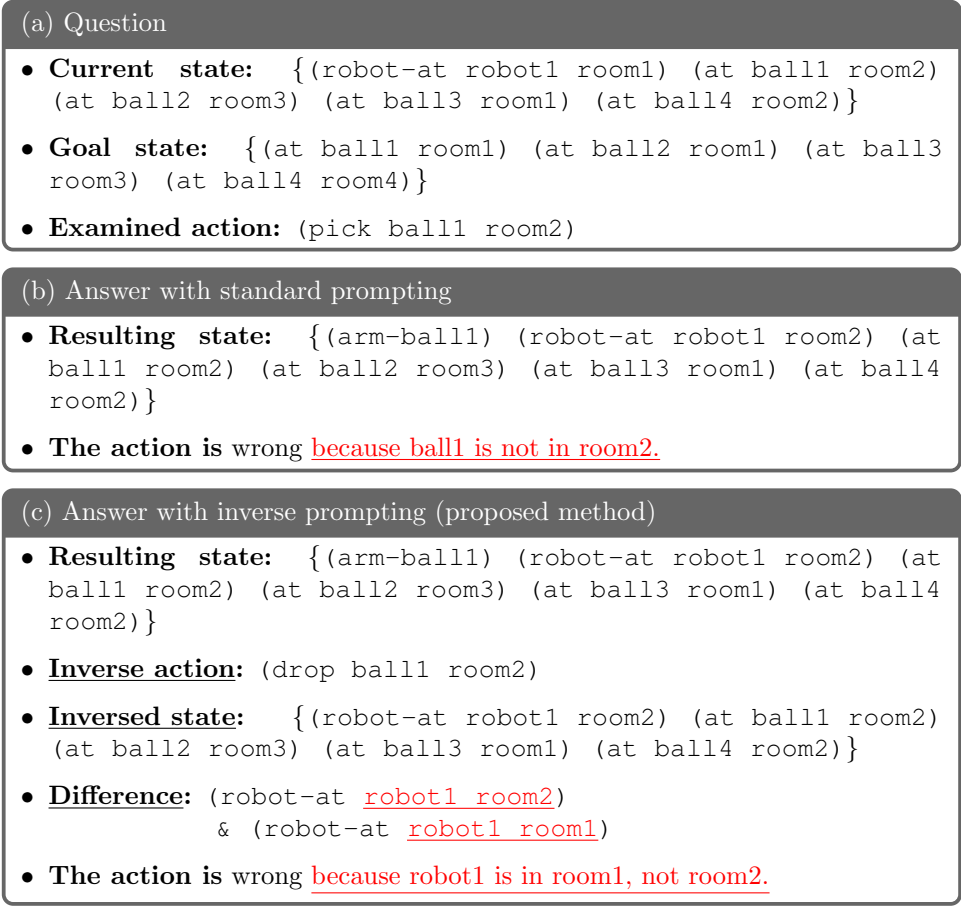


Figure 5.2: Examples of the generated feedback with the standard prompting and the proposed inverse prompting under the Ballmoving domain, given the question in (a). The text underlined in red indicates the groundings for the final answer, while the text underlined in black highlights the reasoning steps in our approach.

able LLM to identify and correct its mistakes, inspired by human problem-solving techniques [90]. However, they simply adopt a self-correction method without considering the structured or symbolic characteristics of robot task planning. As shown in Figure 5.2(b), they provide an explanation for the infeasibility of the action, but they lack thorough groundings or steps that

support a clear reasoning process. It is worth noting that existing single-step reasoning processes often fall short in long-horizon planning tasks, as demonstrated in [91]. It can potentially increase the likelihood of generating a less coherent response and degrade the capability of the feedback as a validator. Consequently, the generated feedback is more prone to failures in correction, being both plausible-sounding and inaccurate.

In this work, we introduce a self-corrective planning approach with inverse prompting, called InversePrompt, to incorporate multiple reasoning steps, inspired by the inversion principle in mathematical verification [92]. The inverse prompting is designed to generate the conclusion with three reasoning steps, as highlighted by the black underline in Figure 5.2(c). It firstly produces inverse actions and the corresponding states. Note that the inverse action can be generated based on the symbolic and linguistic characteristics of the action language. It then verifies whether the inverted state can return to the original state. The result of an operation, such as $x + y = z$, is validated by reversing it, for example, $z - y = x$. In this context, the inverse action and the original state correspond to the role of $-y$ and x , respectively. The proposed method easily generates more detailed and accurate feedback by analyzing the difference between the inverted and the original states. In this way, the proposed InversePrompt effectively enhances the reliability of the feedback while validating the logical flow of the generated plans.

We validate the proposed method under extensive benchmark scenarios, including Ballmoving [93], Blocksworld [3], and Cooking [17] environments.

The results show that the proposed method outperforms existing standard prompting and external validator methods by a margin of 17.5% and 4.2% on average, respectively. We also evaluate the proposed method in real-world environments with infeasible plans. The proposed approach demonstrates its effectiveness in error correction, resulting in more successful task completion than existing self-correction methods. The contributions of this section are as follows:

- We propose self-corrective planning of tasks with an inverse prompting strategy, named InversePrompt, enabling an LLM to provide explicit justifications in the reasoning process.
- The proposed approach introduces a multi-step reasoning process for generating comprehensive feedback and explicitly validating the logical flow of the generated plans with inverse actions and the states.
- Extensive experimental results demonstrate that the proposed approach achieves a 16.3% higher average success rate with fewer attempts than other competitors across benchmark datasets and effectively handles unexpected failures in real experiments.

5.1 Related Works

5.1.1 Corrective Planning

Despite the effectiveness of LLMs in generating task plans, their limitations become evident when the generated plans are occasionally not feasi-

ble in practice [82]. To refine the original task plans, existing works mostly adopt external validators such as predefined rules [18, 83, 84] or additional LLMs [7]. For example, an iterative prompting strategy has been proposed to employ a rule-based verifier as a syntax checker and static analyzer [83]. A multi-level feedback loop has also been introduced using predefined error types [84]. Corrective prompting has been used to address execution failures caused by precondition violations [18]. In addition, multimodal hierarchical summaries of execution histories have been proposed to detect and correct failures with additional LLMs [7].

Recently, self-correction methods have been adopted in robot task planning, where the planner identifies and refines errors by itself [17, 89]. These works using LLMs for error detection and correction rely on a naïve reasoning process through standard prompting, directly answering feasibility or potential problems with a single reasoning step. The absence of thorough reasoning or explicit justifications for responses can lead to failures in detecting errors or the generation of unhelpful feedback for correction. In this work, we introduce an inverse prompting strategy for the self-correction process that involves a multi-step reasoning approach over generated task plans and explicit groundings through inverse actions.

5.2 Proposed Method

5.2.1 Framework

In this section, we propose a method for enabling an LLM to self-correct errors it produces, including both self-detection and self-rectification, without the need for extra resources [7, 18, 83, 84]. As illustrated in Figure 5.3, we first translate the given task goal described in natural language into a PDDL formulation [21] using an LLM, where actions are predefined with their corresponding robot skills [3, 17]. This formulation consists of the initial state s_0 and the goal state s_{goal} , representing the relevant environmental information. Then, the LLM generates a sequence of task plans $P = \{(s_1, a_1), \dots, (s_T, a_T)\}$, where the goal state can be achieved as $s_{goal} = s_T$ starting from s_0 . This sequence comprises sets of actions $\mathcal{A} = \{a_1, \dots, a_T\}$ and states $\mathcal{S} = \{s_1, \dots, s_T\}$. The state transitions can be represented as

$$s_0 + a_1 = s_1, \dots, s_{T-1} + a_T = s_T, \quad (5.1)$$

where the addition operation symbolically represents the transformation from one state to the next through an action.

The generated plan P undergoes a self-correction process, in which the LLM evaluates the feasibility and correctness of the actions and states itself. If the plan is found to be infeasible, feedback is generated to rectify errors in the re-planning phase. This self-correction process with re-planning is repeated until either a feasible action sequence is identified or a predefined

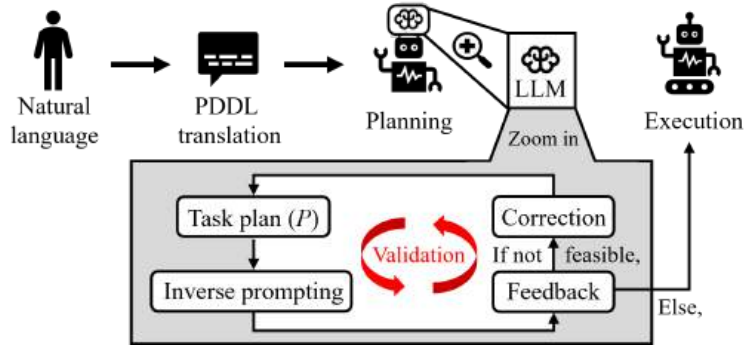


Figure 5.3: Graphical illustration of the proposed overall process.

number of repetitions is reached. We introduce an inverse prompting strategy, called InversePrompt, to enable the LLM to produce explicit feedback with multi-step reasoning during the self-correction process. It is designed to generate inverse actions and evaluate whether the current environment can be restored through the inverse actions, providing justifications for the responses and explicitly validating the logical flow of plans.

Note that we iteratively generate and validate each step of the plan based on the current state information, allowing us to address unexpected failures that often occur in real-world scenarios. The actions and states generated from the self-correction process can be compared with the actual state observed after the execution of the robot. This comparison determines whether the original plan needs to be refined and guides the subsequent execution.

5.2.2 Inverse Prompting

To facilitate the generation of justifications within its responses during self-correction by an LLM, we introduce an inverse prompting strategy. This approach aims to enhance the capability of an LLM to engage in multi-step reasoning, in contrast to the previous methods that typically perform the reasoning process in a single step. It involves three primary steps: 1) verifying the generated state, 2) applying inverse actions, and 3) confirming the goal state. Note that a key difference of this approach compared to existing methods [17, 89] lies in the reasoning by the second step, validation with inverse actions.

In the first step, state verification, the state s_t for each action a_t at time step t in the generated sequence is validated during the planning phase. This step ensures that the generated action is appropriate for achieving the desired state and verifies whether the resulting state is accurate based on the executed action. Second, the inverse action is generated using inverse prompting and applied to the validated state from the first step, resulting in an inverted state. The feasibility of the action is evaluated by comparing the inverted state with the original state. Last, the goal state verification confirms whether the final state s_T matches the pre-determined goal state s_{goal} .

In task planning, particularly described in the PDDL formulation, actions such as “pick” and “putdown” are inverses of each other, and so are “stack” and “unstack.” The states that result from these actions have spe-

cific consequences. For example, in the Blocksworld domain [3], which involves stacking blocks into a specified order, after the “pick” action, the object should no longer be on the table, while after a “putdown” action, it is placed back on the table. Considering these aspects, our proposed method assesses the feasibility of a given plan by verifying whether the environment can return to its original state after performing the inverse action.

The inverse prompting involves three steps to produce the final response with clear justifications. It firstly generates the inverse of a given action, $(a_t)^{-1}$, and then applies it to the current state to verify the possibility of returning to the original state. If executing the inverse action returns to the original state as $s_t + (a_t)^{-1} = s_{t-1}$, the action is considered feasible. Conversely, if it fails to return as $s_t + (a_t)^{-1} \neq s_{t-1}$, it provides feedback to correct the errors. To this end, the LLM compares the original state s_{t-1} with the inverted state after applying the inverse action $\hat{s}_{t-1} = s_t + (a_t)^{-1}$. The difference is used as feedback to guide the re-planning phase. This feedback generated from comparing the two states allows for more explicit corrections for re-planning. Additionally, the reasoning steps allow the LLM to produce richer feedback while progressively generating its justifications and validating the logical flow of plans during self-correction.

Note that during the execution phase, the progress of execution can be monitored through self-correction by comparing the states in the plan $\mathcal{S} = \{s_1, \dots, s_T\}$ with the observed states. If discrepancies arise between the planned and observed states, these differences are utilized as feedback to the

planner. This process allows the planner to revise infeasible plans, guiding the development of a more feasible and logically coherent plan.

5.3 Experiments

We validated the effectiveness of the proposed method, InversePrompt, with an extensive set of experiments conducted in benchmark environments (Section 5.3.1) and real-world settings (Section 5.3.2). The experiments were designed to assess two objectives: 1) the impact of the proposed inverse prompting in self-correction across scenarios and models and 2) the capability to handle unexpected situations during execution by detecting and refining the plan accordingly.

5.3.1 Benchmark Experiments

Setup. We conducted experiments using three widely adopted benchmarks, Ballmoving [93], Blocksworld [3], and Cooking [17]. In the Ballmoving scenario, the objective is to redistribute N balls, initially placed randomly across four rooms, into their goal locations with the constraint that only one ball can be moved at a time. The Blocksworld scenario requires stacking N blocks, initially placed randomly on the table, into a specified order by moving a single block at a time. The Cooking scenario involves distributing up to six different ingredients into N pots according to the recipes. If an ingredient is picked, it can then be distributed into several pots. We generated 20 randomized test cases for each environment, each with a distinct

initial state and a goal state. Following [17], we evaluated the performance on scenarios of varying complexity by setting N to 3 and 4, respectively. The mean lengths of task sequences for the scenarios with $N = 3$ and $N = 4$ are 10 and 13 for Ballmoving, 5 and 8 for Blocksworld, and 20 and 23 for Cooking, respectively.

We utilized two models to verify the applicability of the proposed method: GPT-4o-mini [64], with a model size of approximately 8 billion parameters, and Gemini-1.5-Flash [94], which has a larger model size than GPT-4o-mini although details are not publicly disclosed. For GPT-4o-mini, we set the temperature, which controls the randomness of the response, to 0 for translation and incremented it by 0.1 up to 0.4 with each refinement step in the self-correction process, including planning and validation. For Gemini-1.5-Flash, we set the temperature to 1 for translation and increased it incrementally by 0.1 up to 1.4 with each refinement step during planning, and 2 for validation. We applied few-shot in-context learning [6], a widely used technique, by providing prompt examples to guide the response of LLM. The success or failure of the generated plan was assessed using a rule-based simulation system used in [17]. The maximum number of refinement steps was set to 10.

Comparison methods. We conducted comparative experiments with three types of approaches. Firstly, we compared the proposed method to the baseline approach that does not employ any validator [1,2], named No validator,

performing a single planning stage with an LLM. Secondly, we employed a rule-based external validator that evaluates pre-defined conditions for each action [18, 83, 84]. It detects errors and generates feedback based on pre-defined rules. Lastly, we compared with the self-correction approach using a standard prompting strategy [17, 89].

Results. As in Table 5.1, we summarize the comparison results on the Ball-moving, Blocksworld, and Cooking scenarios using the two models. We can observe that InversePrompt significantly outperforms other methods, achieving average success rates of 78% and 89% with GPT-4o-mini and Gemini-1.5-Flash, respectively. This represents an improvement of 24% and 30% over the baseline that does not employ a validator. Compared to the external validator, the proposed approach achieves up to 8% higher average success rates across models and scenarios. It indicates that its rules for error detection and feedback, although strictly constructed by human experts, may be less adaptable or interpretable. Additionally, the method employing standard prompting in self-correction exhibits, on average, a 17% lower accuracy compared to our InversePrompt for all scenarios and models. The proposed inverse prompting method enables the planner to complete more tasks through the presented self-correction.

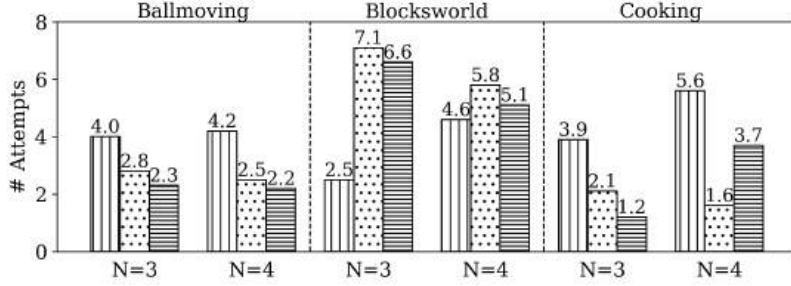
Table 5.1: Comparison results of task success rates across various scenarios and models. Self-corr. w/o IP denotes the self-correction method without the proposed inverse prompting (IP). The bold text indicates the best results.

Scenario	GPT-4o-mini				Gemini-1.5-Flash			
	No Validator	External Validator	Self-corr. w/o IP	InversePrompt (Ours)	No Validator	External Validator	Self-corr. w/o IP	InversePrompt (Ours)
Ballmoving	$N=3$	70%	85%	100%	20%	90%	70%	100%
	$N=4$	50%	75%	100%	25%	55%	25%	50%
Blocksworld	$N=3$	75%	90%	90%	70%	95%	85%	95%
	$N=4$	55%	60%	50%	85%	100%	85%	100%
Cooking	$N=3$	45%	65%	70%	80%	95%	80%	90%
	$N=4$	30%	45%	45%	75%	100%	70%	100%
Average		54%	70%	78%	59%	89%	72%	89%

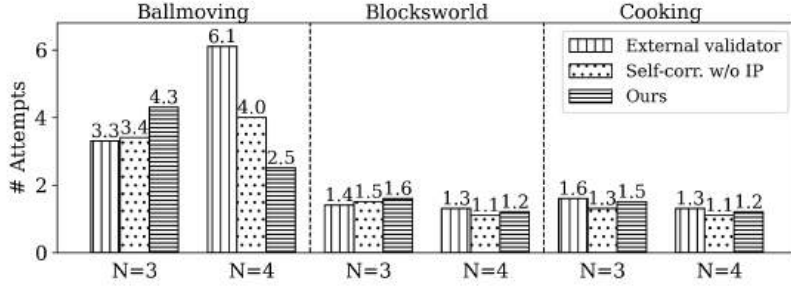
Table 5.2: Performance of the self-correction process.

Scenario		GPT-4o-mini		Gemini-1.5-Flash	
		Self-corr. w/o IP	Ours	Self-corr. w/o IP	Ours
Ballmoving	$N=3$	80%	100%	70%	85%
	$N=4$	80%	100%	25%	55%
Blocksworld	$N=3$	30%	35%	90%	100%
	$N=4$	60%	60%	85%	100%
Cooking	$N=3$	55%	70%	80%	85%
	$N=4$	30%	35%	85%	100%
Average		56%	67%	73%	88%

We assessed the performance of the self-correction process to determine whether it accurately identified successes and failures in Table 5.2. The accuracy was computed as $(TP + TN)/M \times 100$, where TP and TN denote the number of true positives (identifying an actual successful plan as success) and true negatives (classifying a failed plan as failure), respectively, and M is the number of test cases. Our method applying inverse prompting demonstrates approximately 11% and 16% higher accuracy compared to the standard prompting approach without the proposed inverse prompting, for the GPT-4o-mini and Gemini-1.5-Flash, respectively, across all scenarios. Notably, ours exhibits an impressive performance improvement of 30% in the Ballmoving scenario with four balls on Gemini-1.5-Flash compared to the standard reasoning process. This improvement indicates that the proposed inverse prompting method enhances the ability of LLM, improving the self-correction process. In contrast, the method without inverse prompting often fails to detect errors, mistakenly classifying them and resulting in lower



(a) GPT-4o-mini



(b) Gemini-1.5-Flash

Figure 5.4: The number of attempts for correction.

performance.

We also measured the number of attempts for the correction, as plotted in Figure 5.4. If the result from each correction step is infeasible, a new attempt to refine the original plan is initiated to revise it. In most scenarios, the proposed method requires the fewest refinement attempts while achieving the highest performance. For Ballmoving and Cooking, fewer attempts were required compared to those using external validators. It suggests that the feedback generated by InversePropmt is more interpretable and accurate than predefined error types and feedback for re-planning. The self-correction method without the proposed inverse prompting occasionally shows fewer

attempts in Cooking and Blocksworld due to its failure to detect errors. As a result, it fails to make further attempts to correct the previously flawed plan.

Figure 5.2 illustrates selected examples of feedback generated by the self-correction process using the proposed inverse prompting and standard prompting methods for Ballmoving. When employing standard prompting, we often encounter instances of undetected errors and the generation of incorrect feedback. Specifically, Figure 5.2 shows that standard prompting often produced plausible yet incorrect feedback, such as identifying the incorrect location of *ball1* instead of the position of *robot1*. In contrast, the proposed inverse prompting method with its reasoning steps provided detailed feedback on the exact location of the *robot1* and offered clear justifications for errors in the previously generated plan.

5.3.2 Real-World Experiments

Setup. We evaluated the proposed method using a 7-DOF Franka Emika Panda robot arm in both the Blocksworld and Cooking scenarios. In the Blocksworld scenario [3], the robot is required to stack four blocks in the specified order. In the Cooking scenario [17], the robot aims to complete various salad recipes by adding ingredients such as carrots, raisins, mushrooms, granola, and macadamias into three pots, initially containing only green vegetables. We validated the feasibility of the generated action sequence in each environment state through the self-correction process. If its

results indicate an infeasible action sequence, we replanned it based on the generated feedback. Otherwise, we executed the robot action for that sequence. To verify the effectiveness of the proposed inverse prompting approach in the self-correction process, we compared it with existing methods that employ standard prompting [17, 89].

Results. Figure 5.5 shows the actual action sequence performed by the robot. Note that *(action x y)* means that the *action* is performed on the *x* with respect to *y* (sometimes *y* is omitted). In the Blocksworld, the action initially generated by the planner, *(Unstack B4 B3)*, is infeasible in the current state. Ours provided a clear explanation with detailed justification for its infeasibility, resulting in the correctly revised action sequence and successful completion of the task. In contrast, the standard prompting method detected the error but produced unhelpful feedback, leading to the infeasible subsequent plan, *(Pickup B1)*. Afterward, the self-correction process failed to detect this error, resulting in task failure due to a collision with B4, B2, and the robot.

In Cooking, InversePrompt provided detailed feedback on the third plan *(Pick granola)* but non-executable, specifying that the arm is holding a mushroom, facilitating successful re-planning and task completion by the robot. In contrast, the self-correction method without inverse prompting failed to detect the incorrectly generated plan, *(Pick granola)*, mistakenly validating it as a correct action. As a result, the robot executed the action,

leading to task failure due to a collision with the mushroom in the red box.

5.4 Discussion

In this chapter, we have introduced self-corrective knowledge refinement framework with an inverse prompting strategy, named InversePrompt, which incorporates multi-step reasoning to provide explicit justifications for feedback. It first generates the inverse action of the produced action plan to examine the feasibility and logical flow of the generated action sequence. Then, it analyzes the inverted state resulting from the inverse action and the original state to generate feedback. Comprehensive experimental results on benchmark datasets and real-world scenarios demonstrate that the proposed method outperforms existing LLM-based competitors in task completion, self-correction, and feedback generation. By providing more accurate and informative feedback, it achieves a higher success rate while requiring fewer refinement steps.

6. Environment-Aware Knowledge

Alignment through Test-Time Adaptation

Vision-language-action models (VLAs) have recently shown remarkable progress in enabling robots to interpret natural language instructions from humans, understand complex visual scenes, and execute manipulation behaviors [28, 29, 31, 35, 37]. They jointly learn from large-scale robotic multi-modal dataset [30, 32, 95–98], including images, language instructions, robot sensory data, and robot demonstrations. As a result, VLAs can integrate the capability of perception, reasoning, and action generation into an end-to-end policy, highlighting their potential as a scalable and flexible foundation for generalist robot learning [25, 26].

Despite their success, VLAs still suffer from critical limitations when deployed in new environments [28, 29, 41]. In real-world deployment, visual perturbations such as background texture, lighting condition, and camera viewpoints can readily arise and deviate from the training distribution, as illustrated in Figure 6.1. Results from LIBERO-Plus show that existing VLAs are highly sensitive to these perturbations, with average success rates dropping from around 95% to below 30% [68]. Consistently, we observe in our environment that even minor appearance shifts can severely degrade

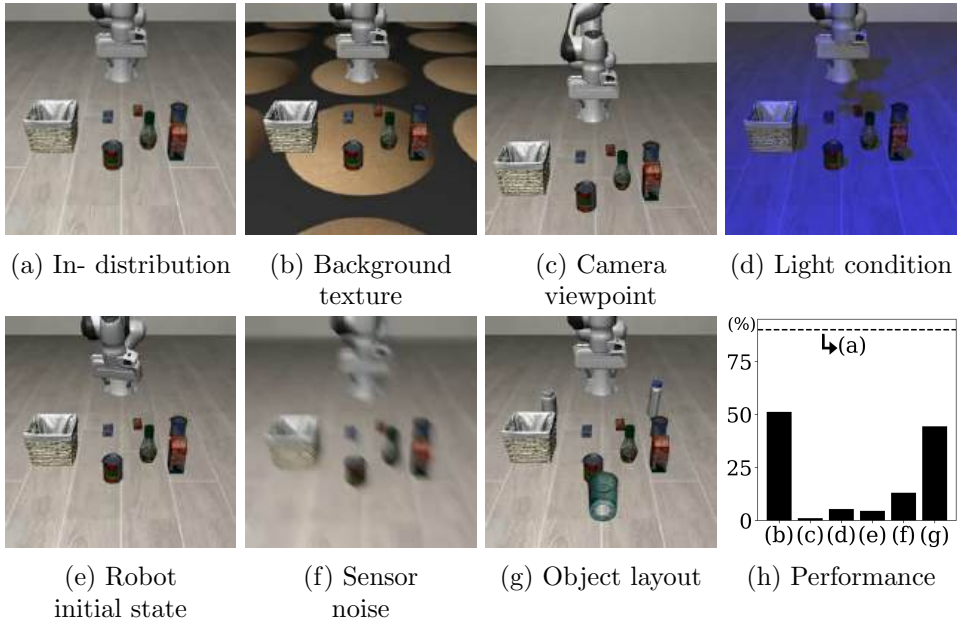


Figure 6.1: Examples of various visual perturbation types in LIBERO-Plus [68], along with their performance on OpenVLA [28]. Original performance without visual perturbation, exemplified by (a), is plotted as a dashed line in (h).

policy performance, yielding an average 70% drop across the six perturbation categories, as in Figure 6.1(h).

To address these problems, most recent works have mainly focused on advancing the training recipe. A common direction is to increase the scale and diversity of demonstrations collected across varied scenes, objects, and sensing conditions [28, 29, 41–44]. A broad range of observations and manipulation behaviors helps the model to learn perception–action relationships that generalize more consistently in novel environments. Another direction is to enhance the reasoning capability of the underlying vision-language backbone [31, 43, 45–48]. Reasoning mechanism enables VLAs to generate

grounded intermediate representations (e.g., object relations, sub-task decomposition), allowing them to leverage the semantic and compositional capabilities of pre-trained large-language models and visual-language models.

While these two directions offer meaningful benefits, they aim to improve the capability of VLAs through extensive training, resulting in substantial additional data collection and computational resources. Furthermore, these advances typically strengthen visual understanding, rather than explicitly addressing distribution shifts at deployment. Consequently, real-world environments vary widely and may include conditions not covered during training, which can lead to inconsistent action behaviors under the resulting distribution shifts, as illustrated in Figure 6.2(a) and Figure 6.2(b). These considerations motivate investigating complementary approaches that enable models to adapt using information available at test time, allowing more reliable operation under unseen conditions.

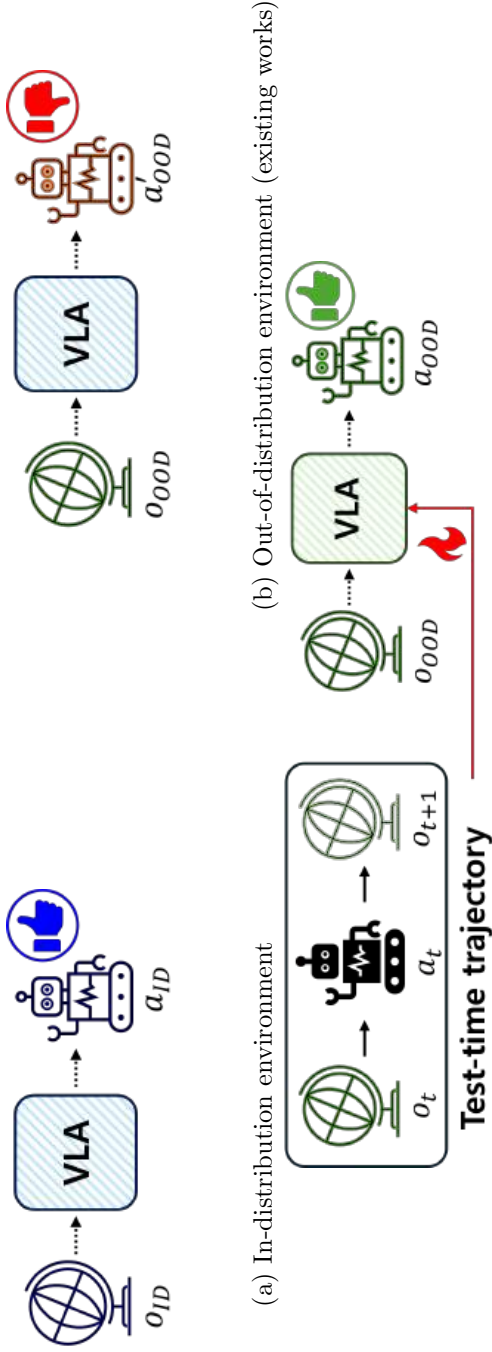


Figure 6.2: Comparison of VLAs under in-distribution and out-of-distribution environments. Blue denotes the in-distribution training domain, green indicates the out-of-distribution domain at test time, and orange represents inconsistent action outputs induced by distribution shifts. (a) In an in-distribution setting, observations align with the training distribution of the VLA, resulting in consistent visual-action representations. (b) Under an out-of-distribution environment, visually shifted observations lead to mismatches between visual inputs and action representations in the pre-trained VLA. (c) Our method effectively adapts the pre-trained VLA using a test-time trajectory in the shifted environment, mitigating visual-action inconsistencies.

To adapt the pre-trained model during deployment without extensive training, test-time adaptation (TTA) provides a natural solution across various domains [99–101]. It adjusts the model parameters using only the inputs available at test time, typically optimizing self-supervised objectives that encourage confident predictions with entropy minimization, consistency regularization, or pseudo-labeling [102–105]. Despite its potential benefits, applying test-time adaptation to VLAs poses additional challenges due to their multi-modal characteristics. In particular, VLAs depend on tight alignment across vision, language, and action modalities. As a result, even a modest visual distribution shift can easily disrupt this alignment and compromise downstream action semantics. However, prior TTA approaches in robotics [99, 106] do not consider the alignment of multi-modal representations, only relying on feedback or entropy of predictions as output-level supervision signals. Furthermore, addressing such cross-modal misalignment at test time requires supervision signals at the representation-level that reflect whether actions remain semantically grounded in both the instruction and the scene. Yet, although robots can obtain rich raw sensory observations at deployment, it remains challenging to derive test-time supervision that explicitly captures cross-modal semantic grounding among vision, language, and action, since existing TTA objectives are largely agnostic to these semantics.

To overcome these challenges, we introduce a **Test-time Trajectory-based Adaptation** framework for **VLAs**, named **TTA-VLA**. The goal of TTA-

VLA is to achieve environment-aware knowledge alignment, where the pre-trained VLA adapts its visual–action representations to observations from the deployment environment while preserving task semantics through language guidance. It exploits visual observations collected from a trajectory in the new deployment settings to explicitly align visual and action representations, as shown in Figure 6.2c. The key insight behind TTA-VLA is that during training, VLAs learn relationships between input observations and output actions, yielding transition patterns in which changes in visual observations induce corresponding action transitions over time. We leverage this temporal consistency between visual and action transitions in their latent representations as a self-supervised signal for test-time adaptation. Under visual domain shifts, however, the pre-trained VLA may fail to preserve its original observation–action relationship, leading to reduced visual–action temporal consistency such that visual changes no longer induce appropriate action transitions. To address this, we extract transition representations from consecutive visual and action embeddings along a test-time trajectory as environment-specific information. Then, we align these transition representations with the task instruction, which serves as a language-conditioned semantic reference, to reduce visual–action temporal inconsistencies induced by unseen visual shifts. This alignment grounds the self-supervised adaptation signal in vision, language, and action semantics, encouraging both visual and action transitions to remain semantically consistent with the intended task. In this way, the proposed method provides a simple yet effective

test-time adaptation that restores disrupted visual–action alignment while maintaining semantic stability under distribution shifts, without requiring supervision from experts or computational overhead. This formulation is not tailored to a specific VLA architecture, but instead addresses test-time adaptation as a general challenge arising from cross-modal misalignment between visual and action representations.

We validate the effectiveness of our test-time adaptation framework under a wide range of visual distribution shifts on two representative VLA paradigms: the autoregressive-based OpenVLA [28] and the flow-matching-based π_0 [29]. For simulation experiments on the LIBERO-Plus [68] with six visual perturbation types, we observe consistent performance improvements over the pre-trained policy across both VLA architectures with an average gain of 14%. To assess the real-world practicality of our approach, we evaluate it under four visual perturbation deployment scenarios, achieving an average success rate improvement of 33%.

6.1 Related Works

6.1.1 Test-Time Strategies

Test-time strategies aim to improve model robustness or performance during inference without requiring full retraining. A representative direction is test-time adaptation (TTA), which mitigates distribution shift by adjusting the model behavior at test time using only unlabeled test-time data.

This is typically achieved by optimizing self-supervised objectives that encourage consistent and confident predictions, thereby avoiding costly retraining or additional supervision [107, 108]. TTA has been widely investigated for maintaining feature consistency under distribution shift with entropy minimization [101, 104] or batch-norm statistics adaptation [105, 109]. For vision-language models, prior works explore optimizing text prompts using augmented images [100, 102] or employing self-supervised proxy tasks [103, 110]. In robotics, only a few studies have explored TTA, primarily in visual-language navigation, using entropy minimization of action predictions [106] or binary feedback-based reinforcement learning [99], which exploit output-level signals rather than the internal latent representations of the model. However, TTA for vision-language-action models remains unexplored, despite their susceptibility to visual distribution shifts.

Another test-time strategy is test-time scaling, which improves inference performance by increasing test-time computation through multiple candidate generation, search, or verification-based selection [111–113]. While test-time scaling can improve output quality without modifying the model, it does not directly adapt the model to the current test distribution and often requires repeated candidate generation and selection at each decision step [113, 114]. This limitation is particularly important for real-time robot execution, where performance degradation under visual distribution shifts is caused by a mismatch between the current observation and the pretrained policy rather than insufficient inference-time search. Therefore, test-time adaptation is

more suitable for addressing visual distribution shifts in VLAs because it directly adjusts the model behavior to the current test condition.

In this work, we introduce test-time adaptation of VLAs by leveraging a test-time trajectory to self-supervise the alignment between visual and action representations disrupted by visual distribution shifts. To stabilize this alignment with task semantics, we incorporate the task instruction as a language-conditioned reference, encouraging visual and action features to remain semantically consistent at the feature-level.

6.2 Proposed Method

6.2.1 Problem Statement: Test-Time Adaptation of Vision Language Action Models

Our goal is to enable vision-language-action models (VLAs) to maintain reliable action generation under environmental distribution shifts by aligning pretrained visual-action knowledge with observations from the target environment using only a test-time trajectory. Typically, VLAs, initialized from pre-trained VLMs, enable general-purpose robot control by taking sensory observations and a task instruction as input and producing low-level actions as output. VLAs are trained on a large-scale dataset of expert demonstrations $\mathcal{D}_{train} = \{(o_t, i_t, a_t)\}_{t=1}^N$, where o_t , i_t , and a_t at time step t denote sensory observation, natural-language task instruction, and its corresponding expert action, respectively. Note that the observation is an RGB image, fol-

lowing the standard VLA setup [28]. The image is first encoded into a visual feature through the visual encoder $\mathcal{F}_v$ as $\bar{z}_t = \mathcal{F}_v(o_t)$ and then transformed by a visual projector $\mathcal{V}$, i.e., $z_t = \mathcal{V}(\bar{z}_t)$, to make it compatible with the language model. The task instruction is also embedded using a text encoder $\mathcal{F}_l$ as $e_t = \mathcal{F}_l(i_t)$. The projected visual feature z_t and language feature e_t are concatenated and fed into the language model, which produces a latent representation u_t that is mapped to the robotic action space. In this way, the VLA model parameterized with θ learns the observation–action relationships by minimizing the supervised objective with the action prediction loss $\mathcal{L}_\theta$:

$$\min_{\theta} \mathbb{E}_{(o_t, i_t, a_t) \sim \mathcal{D}_{train}} \left[\mathcal{L}_\theta(a_t \mid o_t, i_t) \right]. \quad (6.1)$$

When VLAs are deployed in a new environment, they encounter new sensory observations in $\mathcal{D}_{new}$ that differ from the distribution of training data $\mathcal{D}_{train}$ due to the changes in background, lighting, viewpoint, or object appearance. In the context of VLAs, such visual domain shifts do not merely distort visual features as typically in vision tasks, they fundamentally disrupt the tri-modal alignment between vision, language, and action. Specifically, the visual embedding z_t becomes misaligned from the joint embedding space where language instructions e_t and action sequences a_t are anchored, leading to action predictions that are no longer semantically grounded. Therefore, we define the primary challenge of environment-aware knowledge alignment as adapting pretrained visual–action representations to a new environment

while preserving the cross-modal semantic consistency.

At test-time, however, VLAs no longer have access to expert actions, ground-truth labels, or additional demonstrations that could be used to recover this misalignment. The only available signal is incoming sensory observations from the new environment, which may significantly differ from the training distribution. Consequently, we formulate the TTA problem as the following unsupervised optimization:

$$\theta = \underset{\theta_o}{\operatorname{argmin}} \mathbb{E}_{(o_t, i_t) \sim \mathcal{D}_{new}} [\mathcal{L}_{\theta_o}^{\text{TTA}}(o_t, i_t)], \quad (6.2)$$

where $\mathcal{L}_{\theta_o}^{\text{TTA}}$ denotes an objective function for test-time adaptation, and θ_o represents the pre-trained model parameters. This adjusts the pre-trained parameters by leveraging its behavior when exposed to observations from a new distribution.

6.2.2 TTA-VLA: Test-time Trajectory-based Adaptation for Vision Language Action models

To address environment-induced tri-modal misalignment, we propose TTA-VLA, which performs environment-aware alignment by leveraging the temporal structure of observation–action relationships conditioned on language as a self-supervised signal. Specifically, we obtain this signal from a test-time robot trajectory collected in the new environment by arbitrarily executing the robot, from which we capture how the new environment af-

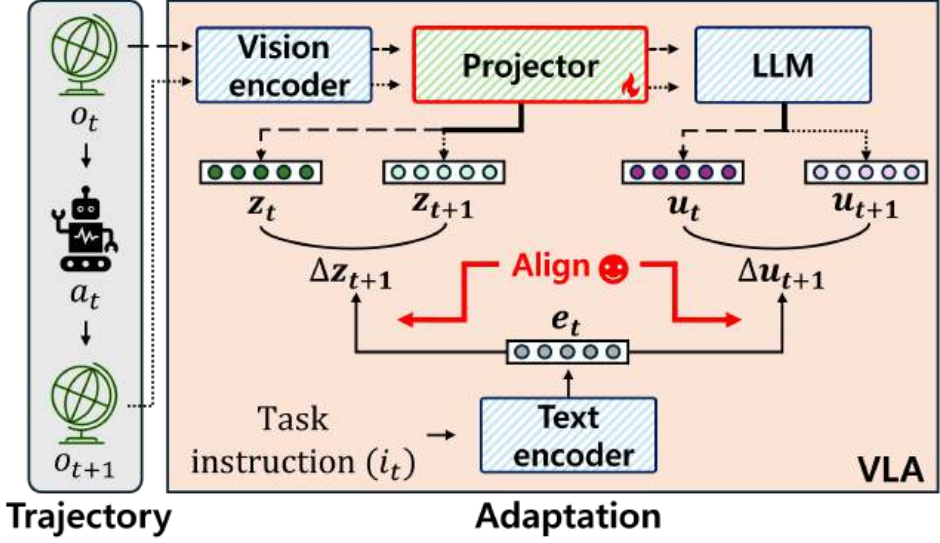


Figure 6.3: Overview. The robot collects observations by executing an trajectory in the new environment. Each observation is converted into visual and action features, z and u , through the pre-trained VLA. Our proposal aligns their transitions, Δz_{t+1} and Δu_{t+1} , while anchoring this alignment to the task-instruction embedding e_t to ensure that both transitions remain semantically consistent with the task i_t .

fects the pre-trained behaviors of the VLA, as depicted in Figure 6.3. Formally, this trajectory can be written as a sequence of observation–action pairs, $(o_t, a_t, o_{t+1}, a_{t+1}, \dots)$. From this trajectory, we encode the observations to extract visual features from the visual projector, z_t and z_{t+1} , and action features from the language model, u_t and u_{t+1} . We then compute their temporal transitions as follows:

$$\Delta z_{t+1} = z_{t+1} - z_t \quad \text{and} \quad \Delta u_{t+1} = u_{t+1} - u_t. \quad (6.3)$$

Our key insight for feature alignment is that VLAs implicitly learn a temporal correspondence between changes in observations and the action representations during training. However, visual domain shifts disrupt this correspondence, motivating the restoration of consistency between the visual transition Δz_{t+1} and the action transition Δu_{t+1} for maintaining reliable behavior of VLAs in the new environment.

To this end, we introduce a transition-based adaptation loss as $\mathcal{L}_{\text{align}}(\Delta z_{t+1}, \Delta u_{t+1})$ to reduce the temporal inconsistency between visual and action feature transitions. We investigate two geometric alignment strategies, angular-based and projection-based, that encourage directional consistency between transitions across modality spaces. The angular-based alignment focuses on matching the orientation of feature transitions by minimizing their cosine distance, inspired by [115]:

$$1 - \frac{\Delta z_{t+1}^\top \Delta u_{t+1}}{|\Delta z_{t+1}|_2 |\Delta u_{t+1}|_2}. \quad (6.4)$$

The projection-based variant projects the action feature transition onto the visual feature transition and penalizes the residual, encouraging action transitions to follow visual transitions, inspired by [116], with a constant for numerical stability ϵ :

$$\left\| \Delta u_{t+1} - \frac{\Delta u_{t+1}^\top \Delta z_{t+1}}{\|\Delta z_{t+1}\|_2^2 + \epsilon} \Delta z_{t+1} \right\|_2^2. \quad (6.5)$$

Their corresponding analyses are provided in Table 6.2.

To further strengthen the language semantic grounding of the adapted representation, we introduce a language-conditioned constraint, which uses the natural-language task description as an environment-invariant reference point. Since the task instruction defines the intended goal and remains unchanged across visual domains, its embedding provides a stable language semantic anchor that guides test-time adaptation while mitigating semantic drift. We incorporate the task instruction as a language semantic anchor by re-centering the visual and action transitions around the text embedding. This gives the following transitions:

$$\Delta \bar{z}_{t+1} = \Delta z_{t+1} - e_t \quad \text{and} \quad \Delta \bar{u}_{t+1} = \Delta u_{t+1} - e_t, \quad (6.6)$$

where e_t denotes the text embedding of the task instruction i_t extracted from the text encoder $\mathcal{F}_l$. Then, we rewrite the objective function as follows:

$$\mathcal{L}_{\theta_o}^{\text{TTA}} = \mathcal{L}_{\text{align}}(\Delta \bar{z}_{t+1}, \Delta \bar{u}_{t+1}). \quad (6.7)$$

By grounding the transition dynamics in the task semantics, the proposal not only stabilizes adaptation but also prevents semantic drift, enabling more reliable behavior under visual domain shifts. During adaptation, only the parameters of the visual projector $\mathcal{V}$ are updated, while all other components of the VLA policy remain frozen, following [117, 118].

6.3 Experiments

6.3.1 Benchmark Experiments

Setup. Dataset. We validate our proposal on LIBERO-Plus [68], which extends LIBERO [79] with systematic and realistic visual perturbations for evaluating test-time adaptation under visual distribution shifts. We assess performance across six axes of visual domain shifts, including background texture, camera viewpoint, lighting condition, robot initial state, sensor noise, and object layout, with examples shown in Figure 6.1. These perturbations expand the original set of 10 tasks to an average of 262, 393, 289, 385, 384, and 404 tasks, respectively, across the object, spatial, and goal variants of the benchmark.

Implementation details. We evaluate the effectiveness of our proposed method on two representative VLA architectures: autoregressive-based OpenVLA [28] and flow-matching-based π_0 [29], which are among the most widely used VLAs. We compare our method against the original VLA policy under two conditions: (i) in-distribution without visual perturbations and (ii) out-of-distribution with visual perturbations to assess the robustness of the originally pre-trained VLAs to visual distribution shifts. In addition, we evaluate two ablated variants of the proposed TTA-VLA: a text-anchored alignment variant defined in Eq. (6.7) and an unanchored alignment variant that directly aligns visual and action transitions without language-conditioned constraint. These ablations analyze the respective contributions of text-

anchored alignment and transition-based adaptation. We used the publicly available pre-trained policy for each backbone and LIBERO variant released by the authors and followed the learning hyperparameters reported in the corresponding papers during adaptation.

Results. *Main results.* We summarize the comparison results on six visual perturbation scenarios in Table 6.1, reporting success rates averaged over object, spatial, and goal variants of LIBERO-Plus. While both OpenVLA and π_0 achieve high performance in the in-distribution setting as 90.0% (first rows) in our environment, their success rates decrease substantially under visual perturbations (second rows), with average drops of 70.8% and 44.8%, respectively, highlighting the sensitivity of pre-trained VLAs to visual distribution shifts. Notably, this degradation is most severe for camera view-point and robot initial state, indicating that geometric and embodiment-related shifts pose a significant challenge. Their performance drop is larger for OpenVLA than for π_0 , likely due to its reliance on a single-camera view. In contrast, background texture and object layout perturbations exhibit the smallest drops among the six types, due to broader exposure to appearance and layout variations during training.

As shown in the third rows, applying our test-time adaptation (w/o language-conditioned constraint) consistently improves robustness across all perturbation types for both backbones. Specifically, our approach improves the average success rate by 9.9% for OpenVLA and 15.3% for π_0 relative

to the originally pre-trained policy. These results indicate that leveraging test-time trajectory provides an effective self-supervision signal for online adaptation under unseen visual perturbations. By formulating adaptation in a transition-based manner, it explicitly enforces temporal alignment between visual observations and actions, thereby reducing performance degradation caused by visual distribution shifts.

The proposed method, TTA-VLA, which further incorporates a language-conditioned constraint (last rows), achieves the best performance across all settings, with improvements of up to 45.4% over the method without adaptation. Notably, the benefits are most pronounced under the most challenging conditions, including camera viewpoint, robot initial state, and sensor noise, where the proposed method achieves substantial additional improvements of 16.3%, 9.8%, and 25.1%, respectively, when averaged across the two backbones. Compared to the results without the language-conditioned constraint in the third rows, its impact can be observed with the consistent improvements of 1.0% on OpenVLA and 1.6% on π_0 across six perturbations. Overall, these results demonstrate that our transition-based adaptation for environment-aware knowledge alignment incorporating language-conditioned constraints further helps align visual and action representations with the intended task semantics, encouraging both modalities to remain semantically consistent throughout test-time execution under severe visual distribution shifts.

Table 6.1: Comparison results of task success rates on LIBERO-Plus across six visual perturbation scenarios. ‘Perturb.’, ‘Adapt.’, and ‘Text’ denote visual perturbation, test-time adaptation, and language-conditioned constraint, respectively. Rows highlighted in blue represent our proposed method, TTA-VLA.

Perturb.	Adapt.	Text	Background texture	Camera viewpoint	Light condition	Robot initial state	Sensor noise	Object layout	Avg.
OpenVLA									
In-distribution			90.0%						
✓			46.4%	1.2%	11.4%	3.7%	5.9%	46.4%	90.0%
✓	✓		64.8%	2.4%	25.9%	11.0%	9.5%	61.2%	19.2%
✓	✓	✓	66.3%	2.7%	26.6%	11.3%	10.7%	62.9%	29.1%
π_0									
In-distribution			90.0%						
✓			79.8%	15.1%	82.5%	5.8%	15.6%	72.6%	90.0%
✓	✓		81.1%	45.1%	85.6%	16.8%	60.5%	74.1%	45.2%
✓	✓	✓	83.1%	46.2%	88.5%	17.7%	61.0%	75.9%	60.5%
									62.1%

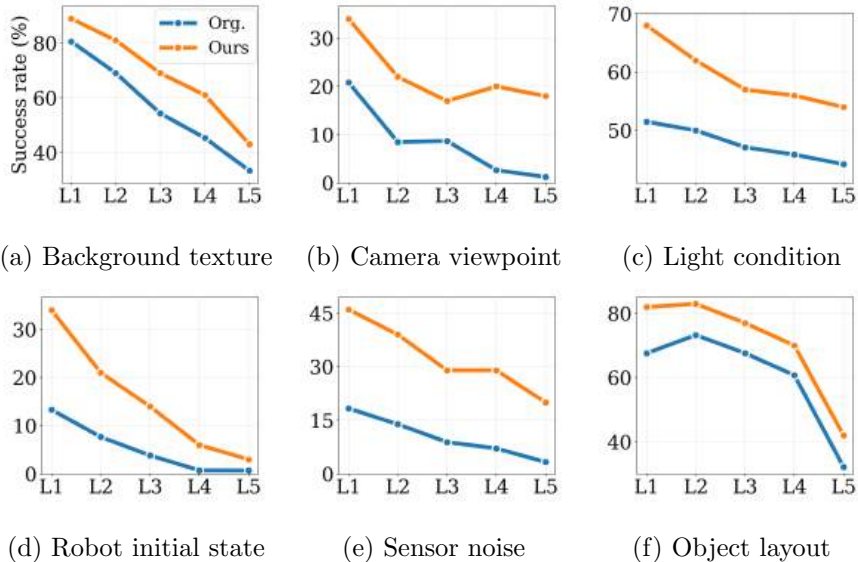


Figure 6.4: Task success rates according to task difficulty under visual perturbations, where difficulty levels in x-axis follow the original LIBERO-Plus. Orange and blue lines denote the original policy and our TTA-VLA policy, respectively.

Performance according to the task difficulty. We analyze the performance of our method with respect to task difficulty, provided in the LIBERO-Plus. In this benchmark, task instances are grouped into five difficulty levels according to how many of the four representative VLAs can successfully complete them. The difficulty ranges from L1, where all models succeed, to L5, where none of them succeed. Figure 6.4 reports task success rates across difficulty levels for each visual perturbation type, averaged over two backbones, comparing the original policy and our adapted policy. As expected, performance consistently decreases from L1 to L5 across all perturbations for both policies. Nevertheless, our method consistently outperforms the original policy across all difficulty levels, achieving average improvements of 16.8% and

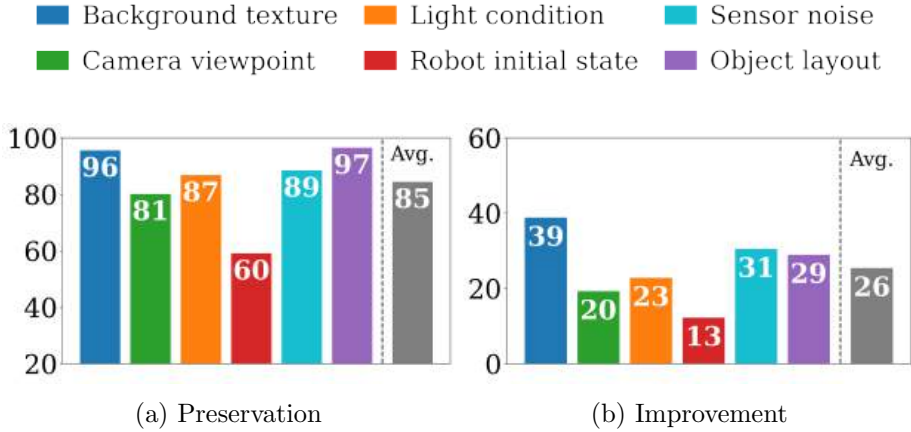


Figure 6.5: Knowledge preservation and improvement under various visual perturbations compared to the original policy.

10.8% at L1 and L5, respectively, across the six perturbations. Notably, for background texture, camera viewpoint, and object layout perturbations, the performance gap between our method and the original policy widens at higher difficulty levels (L3–L5), indicating that our approach is effective in challenging task executions that are inherently difficult for pre-trained policies. These results suggest that TTA-VLA not only preserves performance on easy tasks but also enhances robustness on difficult task instances under visual distribution shifts by leveraging test-time trajectories to mitigate compounded challenges from both task difficulty and visual perturbations.

Knowledge preservation and improvement. To assess the stability and plasticity trade-off of our test-time adaptation framework, we analyze the extent to which successful behaviors of the pre-trained policy are preserved and failed ones are improved under visual distribution shifts. Figure 6.5 presents the measured knowledge preservation and improvement of our adapted pol-

icy θ relative to the original policy θ_o across two models. For a given policy θ , let $\mathcal{S}_\theta$ and $\mathcal{F}_\theta$ denote the sets of tasks that are successfully completed and failed under that policy, respectively. We define Preservation (P) as the percentage of tasks that remain successful after adaptation among those that are successful under the original policy, and Improvement (I) as the percentage of tasks that are successfully completed by the adapted policy among those that fail under the original policy, as follows:

$$P = \frac{|\mathcal{S}_{\theta_o} \cap \mathcal{S}_\theta|}{|\mathcal{S}_{\theta_o}|} \times 100, \quad I = \frac{|\mathcal{F}_{\theta_o} \cap \mathcal{S}_\theta|}{|\mathcal{F}_{\theta_o}|} \times 100. \quad (6.8)$$

Our approach preserves a large portion of the original policy’s successes, achieving an average preservation of 85. At the same time, the proposed method substantially improves previously failed cases, yielding an average improvement of 26. Interestingly, background texture, sensor noise, and object layout perturbations exhibit the highest levels of both preservation and improvement among the six perturbation types. These results indicate that our method effectively balances maintaining existing successful behaviors and recovering failures, demonstrating robust adaptation without sacrificing prior performance under diverse visual distribution shifts.

Variations of alignment strategies. Table 6.2 compares projection-based and angular-based approaches for feature alignment in $\mathcal{L}_{\text{align}}$, reporting the performance gains of the projection-based approach over the angular-based one across six perturbation types on the object, spatial, and goal variants

Table 6.2: Performance improvements with projection-based alignment compared to angular-based alignment in $\mathcal{L}_{\text{align}}$. ‘Text’ indicates language-conditioned constraint.

Text	Object	Spatial	Goal	Avg.
	+ 2.4	+ 2.8	+ 4.9	+ 3.7
✓	+ 2.4	+ 3.4	+ 4.2	+ 3.5

of the benchmark using OpenVLA. More details of the two alignment approaches in Section 6.2.2. Across all settings, the projection-based alignment consistently yields better performance with an average gain of 3.6%, indicating that explicitly suppressing deviated transition components leads to more stable alignment across heterogeneous modal spaces. The angular-based alignment shows competitive but slightly lower results, as it only constrains direction without fully addressing representational discrepancies between modalities. Nevertheless, the performance differences between the two methods are relatively minor compared to the original policy.

Visualizations of rollouts. Figure 6.6 presents comparisons of rollouts between the original policy and TTA-VLA under light condition and sensor noise perturbations. Under light condition, the original policy fails to execute precise manipulation, exhibiting inaccurate grasp poses that prevent stable pickup of the target bowl. In contrast, our method successfully adapts the test-time visual perturbation, producing accurate grasp configurations and completing the task reliably despite the challenging illumination changes. Similarly, under sensor noise, the original policy struggles to localize the target object, leading to incorrect reaching behaviors and task failure. How-

ever, TTA-VLA identifies the target object and executes the pick-and-place sequence correctly, even under severe observation corruption. More rollout visualizations under other perturbation types are provided in the supplementary materials.

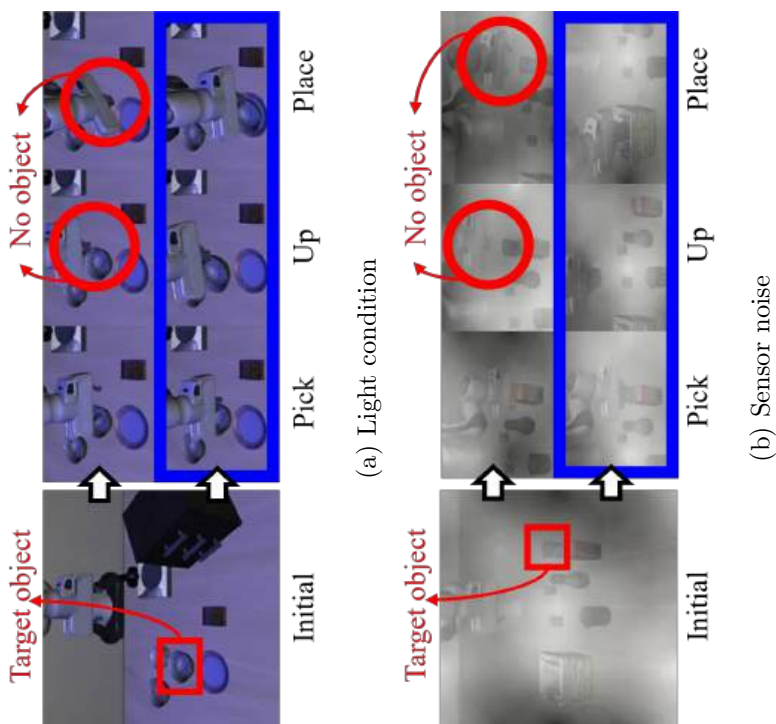


Figure 6.6: Visualizations of rollouts with the original policy (top row) and TTA-VLA (bottom row) on LIBERO-Plus under (a) light condition and (b) sensor noise perturbations. Key failure scenes are highlighted with red circles, while successful executions are highlighted with blue boxes. Red circles typically indicate an attempted grasp with no object.

6.3.2 Real-World Experiments

Setup. *Tasks.* To evaluate robustness while isolating the effects of visual perturbations and adaptation from other factors affecting task performance, we use four fundamental tabletop manipulation tasks representing core manipulation capabilities: pick-and-place, slide, push, and pull. The first task, *Pick up the red block and place it on the plate* (Pick-and-place), requires the robot to accurately localize the target object, grasp it stably, and place it at the designated location. The second task, *slide the green truck to the red car* (Slide), expects the robot to maintain contact with the object and slide it along the surface toward the target. The third task, *push the blue button* (Push), requires the robot to precisely position its end effector and apply a directional pushing motion. The fourth task, *pull out the tissue* (Pull), expects the robot to grasp the tissue and pull it out smoothly without losing contact.



Figure 6.7: Examples of real-world task environments under different visual perturbations for the task of Pick-and-place.

Implementation details. We conducted real-world experiments using a 7-DoF robotic arm Franka Research 3 (FR3) with a two-finger gripper. For the evaluation, we employed OpenVLA using visual input from a frontal third-person camera and the language task instruction. We design four types of visual perturbations, including background texture (Background), camera viewpoint (Camera), robot initial state (Robot), and object layout (Object), as illustrated in Figure 6.7. For Background, the in-distribution setting uses a black background, while five additional backgrounds with different colors are used for evaluation. For Camera, the in-distribution setting uses a frontal third-person view, and four additional camera viewpoints are evaluated. For Robot, the in-distribution setting corresponds to the home pose, while five alternative initial poses are used. For Object, we construct five evaluation settings by introducing new surrounding objects or by changing the positions of existing surrounding objects relative to the in-distribution configuration. We start from a robot home pose except Robot, and objects are placed within a designated black region to guarantee the reachability of the robot.

Results. Quantitative results. We summarize the comparative success rates of the original policy and our method across four real-world scenarios in Table 6.3. To enable fine-grained evaluation, we further decompose the overall success for each task into stage-wise scores, such as approach, grasp, transport, and place. Overall, the original policy is highly sensitive to real-world visual distribution shifts, most notably for Camera with average drop of 73%

and Robot with 64% across all tasks, indicating that mismatches in observation geometry and starting configurations strongly disrupt execution. In contrast, TTA-VLA consistently improves performance across all tasks and perturbation types, yielding an average gain of 33% over the pre-trained policy. In particular, the proposed method shows large gains of 45% and 43% under Camera and Robot, respectively. We also observe consistent gains under Background with 25% and Object with 19%, demonstrating that TTA-VLA remains effective even when the shift is dominated by appearance changes or scene rearrangements. Notably, these gains are consistent across four tabletop real-world tasks that span core manipulation capabilities and varying execution demands (Pick-and-place, Slide, Push, and Pull), suggesting that the benefit of TTA-VLA generalizes beyond a single skill. These gains can be attributed to the proposed transition-based alignment objective, which leverages a test-time trajectory and language-conditioned supervision to restore visual–action inconsistencies induced by real-world distribution shifts.

Table 6.3: Comparison results of task success rates (%) on real-world experiments with four visual perturbation scenarios.

Task	In-distribution	Background		Camera		Robot		Object		Avg.	
		Org.	Ours	Org.	Ours	Org.	Ours	Org.	Ours	Org.	Ours
Pick-and-place	100	50	90	15	45	35	65	70	100	39	75
Slide	100	70	100	25	60	15	60	70	100	45	80
Push	90	90	95	25	70	50	80	90	100	64	86
Pull	90	70	90	0	40	40	90	80	95	48	79
Avg.	95	70	95	22	67	31	74	78	97	50	83

Visualizations of rollouts. We show representative real-world rollouts across all scenarios, comparing the original policy (top row) and TTA-VLA (bottom row) in both successful (Figure 6.8) and failed cases (Figure 6.9). For clearer visualization, we render the scene with reduced opacity while excluding the robot and objects, allowing the motion trajectories to be more easily observed. In the success examples, the original policy exhibits unstable behaviors, such as mislocalizing the target object under background shifts in Slide and being distracted by changes in surrounding objects in Pick-and-place, which leads to missing the target or interacting with the wrong region. In contrast, the proposed method maintains stable, target-centric motion and completes the intended manipulation by restoring visual-action inconsistencies induced by visual distribution shifts at the representation-level. In the failed cases, even when not fully successful, we observe a clear gap between the two methods: in Pull, our method consistently approaches the target object but sometimes fails due to minor final pose misalignment, whereas the original policy drifts to an irrelevant area and abandons the target altogether. Similarly, in Push, the original policy reaches near the target but pushes toward an inaccurate location and fails to finish the task, while our method produces more task-relevant interactions that can complete the task, but with slight residual positioning errors.

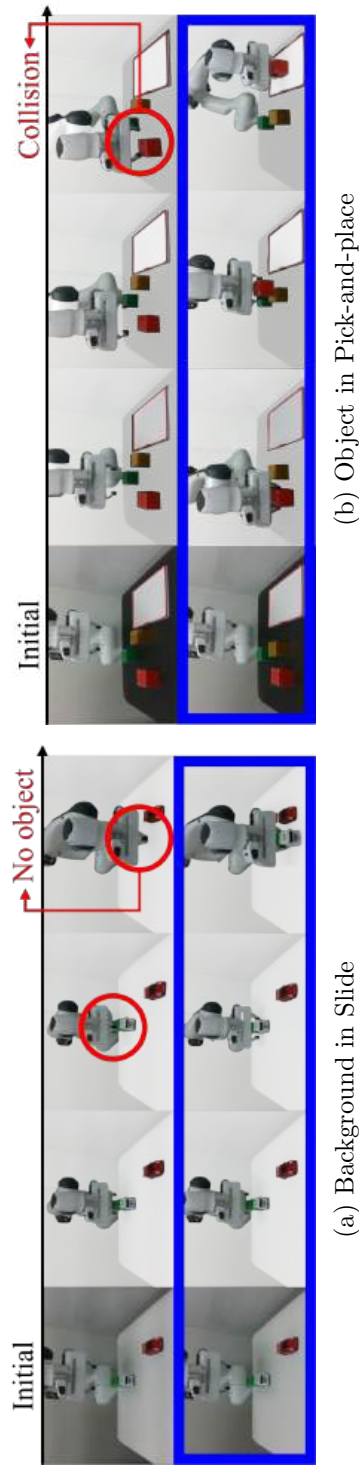
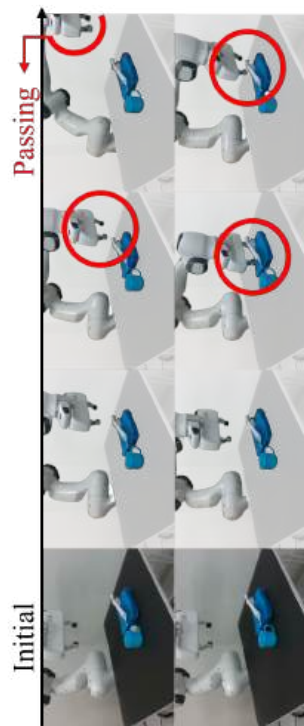
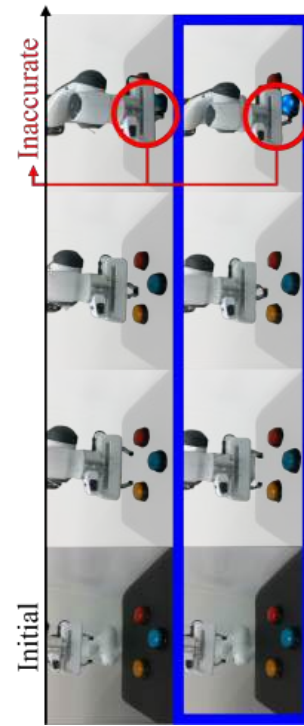


Figure 6.8: Real-world rollouts between the original policy (first row) and our method (second row). Key failure scenes are highlighted with red circles.



(a) Camera in Pull



(b) Robot in Push

Figure 6.9: Failed real-world rollouts. Task completions are highlighted with blue box.

6.4 Discussion

In this chapter, we have addressed the challenge of environment-aware knowledge alignment for vision-language-action models deployed under unseen visual conditions. To this end, we have introduced TTA-VLA, a test-time trajectory-based adaptation framework that leverages visual-action transition consistency conditioned on language as a self-supervised signal, enabling VLAs to adapt using only a small number of test-time observations. To further preserve semantic stability, it anchors transition alignment to the natural-language task instruction, providing an environment-invariant semantic reference that mitigates representational drift. Through extensive experiments on the benchmark and real-world scenarios across various visual perturbations, we demonstrate that our method significantly improves success rates under diverse visual distribution shifts, with two representative VLA backbone paradigms, OpenVLA and π_0 . These results highlight that test-time adaptation, driven by simple yet effective self-supervised signals, is a promising direction for enhancing the robustness of VLAs in reliable deployment.

7. Conclusion

7.1 Summary

This dissertation studied adaptive robot learning with foundation models, focusing on how pretrained knowledge can be adapted to continually improve the reliability of their behavior in changing real-world environments. Although foundation models provide strong generalization across task planning and action policy execution, pretrained knowledge does not always translate into robot behavior that is appropriate for the current context. This dissertation viewed this limitation as a knowledge-to-behavior mismatch and investigated how robots can mitigate this mismatch by adapting the knowledge used for reliable robot behavior generation through interaction with the physical world.

To this end, this dissertation addressed four directions of knowledge adaptation. First, robots should acquire useful knowledge from their experience when pretrained knowledge is insufficient for new situations. Second, robots should remove harmful knowledge when it induces unreliable or undesirable behavior. Third, robots should refine uncertain knowledge when erroneous reasoning leads to unverified decisions. Finally, robots should align situated knowledge when its feasibility depends on a specific environment or

interaction context. Across these directions, this dissertation proposed several methods that span high-level task planning and low-level action policies.

In Chapter 3, this dissertation developed an active knowledge acquisition framework for LLM-based task planning. This chapter addressed how robots can construct and utilize informative experiences generated during deployment. The proposed method uses a contrastive learning strategy that leverages both successful and failed experiences stored in active memory. Successful experiences guide desirable reasoning patterns, while failed experiences help the model recognize and avoid incorrect behaviors. By actively selecting and retaining informative examples within a limited memory budget, this chapter shows how robots can acquire and learn useful knowledge from experience for task planning.

In Chapter 4, this dissertation developed a selective knowledge removal framework through policy unlearning for VLA policies. This chapter addressed the retention of harmful knowledge that can lead to unreliable behavior during policy execution, particularly state hallucination in VLAs. The proposed method analyzes internal policy representations to identify activation patterns associated with hallucinated behavior. By suppressing hallucination-inducing activation patterns while preserving representations necessary for successful task execution, this chapter shows how robots can remove harmful knowledge to improve policy reliability.

In Chapter 5, this dissertation developed a self-corrective knowledge refinement framework through feedback-guided inverse reasoning for LLM-

based task planning. This chapter addressed how robots can detect, analyze, and revise their own erroneous reasoning during execution without relying on external supervision. The proposed method generates inverse actions corresponding to initially generated actions and verifies whether they can restore the system to its original state. By combining backward verification with multi-step reasoning to evaluate the logical coherence of generated plans, this chapter shows how robots can refine erroneous decisions using self-generated feedback and support autonomous plan correction.

In Chapter 6, this dissertation developed an environment-aware knowledge alignment framework through test-time adaptation for VLA policies. This chapter addressed the problem that pretrained visual and action representations can become misaligned when the deployment environment differs from the training environment. The proposed method uses a trajectory collected at test time to realign disrupted visual-action relationships based on temporal transitions. By further using natural-language task instructions as an environment-invariant reference, this chapter shows how robots can align their policies with unseen deployment conditions while mitigating representational drift.

This dissertation validated the proposed methods across both high-level task planning and low-level action policies, demonstrating their effectiveness in benchmark environments and real-world experiments. These results suggest that adaptive robot learning with foundation models should not be understood only as improving generalization from pretrained knowledge.

Rather, it should also involve adapting how knowledge is acquired, removed, refined, and aligned according to experience, feedback, and environmental conditions. Overall, this dissertation provides a step toward foundation-model-based robotic systems that can continually improve, operate reliably, and adapt robustly in dynamic real-world environments.

7.2 Future Directions

This dissertation explored adaptive robot learning as a process in which robots acquire useful knowledge, remove harmful knowledge, refine erroneous decisions, and align situated knowledge with deployment environments. While the proposed methods instantiated these capabilities in separate problem settings, a natural future direction is to develop a unified adaptive robot learning framework in which a robot can dynamically select and coordinate these capabilities according to the source of knowledge-to-behavior mismatch. In such an integrated framework, robots would not simply execute a fixed policy or update a model in isolation. Instead, they would continuously diagnose whether unreliable behavior arises from insufficient, harmful, uncertain, or situated knowledge, and activate the corresponding adaptation process as tasks progress. This closed-loop process would allow adaptive robot learning to move beyond a collection of independent modules toward robotic systems that can selectively perform different adaptation capabilities through sustained interaction with the physical world.

Another important direction is continual learning and unlearning for

robots based on foundation models. The adaptive mechanisms explored in this dissertation were mainly studied within specific tasks, environments, or deployment conditions. However, real-world robots must operate over long periods of time, where they continuously encounter new tasks, changing environments, and evolving requirements. In such settings, robots should be able to acquire and retain useful knowledge while also updating, reorganizing, or removing knowledge that becomes outdated, unreliable, or harmful. Future research should therefore investigate mechanisms that jointly support continual knowledge acquisition and selective forgetting, enabling robots to improve their task knowledge and action policies over time without catastrophic forgetting or the accumulation of undesirable behaviors.

Beyond adapting to physical environments, future adaptive robots should also adapt to humans. In real-world settings, robots often interact with people whose goals, preferences, and intentions are not fully specified through explicit instructions. Therefore, adaptive robot learning should be extended toward human-aware adaptation, where robots infer implicit context from human behavior, task history, language, and environmental cues. Such capabilities would allow robots to personalize their behavior, understand unstated user intentions, and adjust their actions according to human needs and preferences.

In summary, these future directions point toward a broader vision of adaptive robot learning in which robots continuously acquire, remove, refine, and align knowledge across both physical and human-centered environments.

References

- [1] Anthony Brohan, Yevgen Chebotar, Chelsea Finn, Karol Hausman, Alexander Herzog, Daniel Ho, Julian Ibarz, Alex Irpan, Eric Jang, Ryan Julian, et al. Do as i can, not as i say: Grounding language in robotic affordances. In *Conference on robot learning*, pages 287–318. PMLR, 2023.
- [2] Wenlong Huang, Fei Xia, Ted Xiao, Harris Chan, Jacky Liang, Pete Florence, Andy Zeng, Jonathan Tompson, Igor Mordatch, Yevgen Chebotar, et al. Inner monologue: Embodied reasoning through planning with language models. In *Conference on Robot Learning*, pages 1769–1782. PMLR, 2023.
- [3] Bo Liu, Yuqian Jiang, Xiaohan Zhang, Qiang Liu, Shiqi Zhang, Joydeep Biswas, and Peter Stone. Llm+ p: Empowering large language models with optimal planning proficiency. *arXiv preprint arXiv:2304.11477*, 2023.
- [4] Jungkyoo Shin, Jieun Han, SeungJun Kim, Yoonseon Oh, and Eunwoo Kim. Task planning for long-horizon cooking tasks based on large language models. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 13613–13619. IEEE, 2024.
- [5] Roya Firoozi, Johnathan Tucker, Stephen Tian, Anirudha Majumdar, Jiankai Sun, Weiyu Liu, Yuke Zhu, Shuran Song, Ashish Kapoor, Karol Hausman, et al. Foundation models in robotics: Applications, challenges, and the future. *The International Journal of Robotics Research*, page 02783649241281508, 2023.
- [6] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Aspell, et al. Language models are few-shot

- learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.
- [7] Zeyi Liu, Arpit Bahety, and Shuran Song. Reflect: Summarizing robot experiences for failure explanation and correction. In *Conference on Robot Learning*, pages 3468–3484. PMLR, 2023.
 - [8] Lingfeng Sun, Devesh K Jha, Chiori Hori, Siddarth Jain, Radu Corcodel, Xinghao Zhu, Masayoshi Tomizuka, and Diego Romeres. Interactive planning using large language models for partially observable robotic tasks. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 14054–14061. IEEE, 2024.
 - [9] Chan Hee Song, Jiaman Wu, Clayton Washington, Brian M Sadler, Wei-Lun Chao, and Yu Su. Llm-planner: Few-shot grounded planning for embodied agents with large language models. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 2998–3009, 2023.
 - [10] Jacky Liang, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, and Andy Zeng. Code as policies: Language model programs for embodied control. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*, pages 9493–9500. IEEE, 2023.
 - [11] Shyam Sundar Kannan, Vishnunandan LN Venkatesh, and Byung-Cheol Min. Smart-llm: Smart multi-agent robot task planning using large language models. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 12140–12147. IEEE, 2024.
 - [12] Yanjiang Guo, Yen-Jen Wang, Lihan Zha, and Jianyu Chen. Doremi: Grounding language model by detecting and recovering from plan-execution misalignment. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 12124–12131. IEEE, 2024.

- [13] Ishika Singh, Valts Blukis, Arsalan Mousavian, Ankit Goyal, Danfei Xu, Jonathan Tremblay, Dieter Fox, Jesse Thomason, and Animesh Garg. Progprompt: Generating situated robot task plans using large language models. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*, pages 11523–11530. IEEE, 2023.
- [14] Gabriel Sarch, Yue Wu, Michael Tarr, and Katerina Fragkiadaki. Open-ended instructable embodied agents with memory-augmented large language models. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 3468–3500, 2023.
- [15] Lihan Zha, Yuchen Cui, Li-Heng Lin, Minae Kwon, Montserrat Gonzalez Arenas, Andy Zeng, Fei Xia, and Dorsa Sadigh. Distilling and retrieving generalizable knowledge for robot manipulation via language corrections. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 15172–15179. IEEE, 2024.
- [16] Leonard Bärmann, Rainer Kartmann, Fabian Peller-Konrad, Jan Niehues, Alex Waibel, and Tamim Asfour. Incremental learning of humanoid robot behavior from natural interaction and large language models. *Frontiers in Robotics and AI*, 11:1455375, 2024.
- [17] Zhehua Zhou, Jiayang Song, Kunpeng Yao, Zhan Shu, and Lei Ma. Isr-llm: Iterative self-refined large language model for long-horizon sequential task planning. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 2081–2088. IEEE, 2024.
- [18] Shreyas Sundara Raman, Vanya Cohen, Ifrah Idrees, Eric Rosen, Raymond Mooney, Stefanie Tellex, and David Paulius. Cape: Corrective actions from precondition errors using large language models. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 14070–14077. IEEE, 2024.
- [19] Stanislaw Antol, Aishwarya Agrawal, Jiasen Lu, Margaret Mitchell, Dhruv Batra, C Lawrence Zitnick, and Devi Parikh. Vqa: Visual question answering. In *Proceedings of the IEEE international conference on computer vision*, pages 2425–2433, 2015.

- [20] Constructions Aeronautiques, Adele Howe, Craig Knoblock, ISI Drew McDermott, Ashwin Ram, Manuela Veloso, Daniel Weld, David Wilkins Sri, Anthony Barrett, Dave Christianson, et al. Pddl—the planning domain definition language. *Technical Report, Tech. Rep.*, 1998.
- [21] Patrik Haslum, Nir Lipovetzky, Daniele Magazzeni, Christian Muise, Ronald Brachman, Francesca Rossi, and Peter Stone. *An introduction to the planning domain definition language*, volume 13. Springer, 2019.
- [22] Lucas Beyer, Andreas Steiner, André Susano Pinto, Alexander Kolesnikov, Xiao Wang, Daniel Salz, Maxim Neumann, Ibrahim Alabdulmohsin, Michael Tschannen, Emanuele Bugliarello, et al. Paligemma: A versatile 3b vlm for transfer. *arXiv preprint arXiv:2407.07726*, 2024.
- [23] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- [24] Siddharth Karamcheti, Suraj Nair, Ashwin Balakrishna, Percy Liang, Thomas Kollar, and Dorsa Sadigh. Prismatic vlms: Investigating the design space of visually-conditioned language models. In *Forty-first International Conference on Machine Learning*, 2024.
- [25] Yuen Ma, Zixing Song, Yuzheng Zhuang, Jianye Hao, and Irwin King. A survey on vision–language–action models for embodied ai. *IEEE Transactions on Neural Networks and Learning Systems*, 2026.
- [26] Ranjan Sapkota, Yang Cao, Konstantinos I Roumeliotis, and Manoj Karkee. Vision-language-action models: Concepts, progress, applications and challenges. *arXiv preprint arXiv:2505.04769*, 2025.
- [27] Brianna Zitkovich, Tianhe Yu, Sichun Xu, Peng Xu, Ted Xiao, Fei Xia, Jialin Wu, Paul Wohlhart, Stefan Welker, Ayzaan Wahid, et al. Rt-2: Vision-language-action models transfer web knowledge to robotic

- control. In *Conference on Robot Learning*, pages 2165–2183. PMLR, 2023.
- [28] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan P Foster, Pannag R Sanketi, Quan Vuong, et al. Openvla: An open-source vision-language-action model. In *Conference on Robot Learning*, pages 2679–2713. PMLR, 2025.
- [29] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, et al. π_0 : A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.
- [30] Abby O’Neill, Abdul Rehman, Abhiram Maddukuri, Abhishek Gupta, Abhishek Padalkar, Abraham Lee, Acorn Pooley, Agrim Gupta, Ajay Mandlekar, Ajinkya Jain, et al. Open x-embodiment: Robotic learning datasets and rt-x models: Open x-embodiment collaboration 0. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 6892–6903. IEEE, 2024.
- [31] Gemini Robotics Team, Saminda Abeyruwan, Joshua Ainslie, Jean-Baptiste Alayrac, Montserrat Gonzalez Arenas, Travis Armstrong, Ashwin Balakrishna, Robert Baruch, Maria Bauza, Michiel Blokzijl, et al. Gemini robotics: Bringing ai into the physical world. *arXiv preprint arXiv:2503.20020*, 2025.
- [32] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Joseph Dabis, Chelsea Finn, Keerthana Gopalakrishnan, Karol Hausman, Alexander Herzog, Jasmine Hsu, et al. Rt-1: Robotics transformer for real-world control at scale. *Robotics: Science and Systems XIX*, 2023.
- [33] Danny Driess, Fei Xia, Mehdi SM Sajjadi, Corey Lynch, Aakanksha Chowdhery, Brian Ichter, Ayzaan Wahid, Jonathan Tompson, Quan Vuong, Tianhe Yu, et al. Palm-e: an embodied multimodal language

- model. In *Proceedings of the 40th International Conference on Machine Learning*, pages 8469–8488, 2023.
- [34] Octo Model Team, Dibya Ghosh, Homer Walke, Karl Pertsch, Kevin Black, Oier Mees, Sudeep Dasari, Joey Hejna, Tobias Kreiman, Charles Xu, et al. Octo: An open-source generalist robot policy. *arXiv preprint arXiv:2405.12213*, 2024.
 - [35] Suneel Belkhale, Tianli Ding, Ted Xiao, Pierre Sermanet, Quon Vuong, Jonathan Tompson, Yevgen Chebotar, Debidatta Dwibedi, and Dorsa Sadigh. Rt-h: Action hierarchies using language. *arXiv preprint arXiv:2403.01823*, 2024.
 - [36] Karl Pertsch, Kyle Stachowicz, Brian Ichter, Danny Driess, Suraj Nair, Quan Vuong, Oier Mees, Chelsea Finn, and Sergey Levine. Fast: Efficient action tokenization for vision-language-action models. *arXiv preprint arXiv:2501.09747*, 2025.
 - [37] NVIDIA, Johan Bjorck, Nikita Cherniadev, Fernando Castañeda, Xingye Da, Runyu Ding, Linxi ”Jim” Fan, Yu Fang, Dieter Fox, Fengyuan Hu, Spencer Huang, Joel Jang, Zhenyu Jiang, Jan Kautz, Kaushil Kundalia, Lawrence Lao, Zhiqi Li, Zongyu Lin, Kevin Lin, Guilin Liu, Edith Llontop, Loic Magne, Ajay Mandlekar, Avnish Narayan, Soroush Nasiriany, Scott Reed, You Liang Tan, Guanzhi Wang, Zu Wang, Jing Wang, Qi Wang, Jiannan Xiang, Yuqi Xie, Yinzhen Xu, Zhenjia Xu, Seonghyeon Ye, Zhiding Yu, Ao Zhang, Hao Zhang, Yizhou Zhao, Ruijie Zheng, and Yuke Zhu. GR00T N1: An open foundation model for generalist humanoid robots. In *ArXiv Preprint*, March 2025.
 - [38] Songming Liu, Lingxuan Wu, Bangguo Li, Hengkai Tan, Huayu Chen, Zhengyi Wang, Ke Xu, Hang Su, and Jun Zhu. Rdt-1b: a diffusion foundation model for bimanual manipulation. *arXiv preprint arXiv:2410.07864*, 2024.
 - [39] Mustafa Shukor, Dana Aubakirova, Francesco Capuano, Pepijn Kooijmans, Steven Palma, Adil Zouitine, Michel Aractingi, Caroline Pascal,

- Martino Russi, Andres Marafioti, et al. Smolvla: A vision-language-action model for affordable and efficient robotics. *arXiv preprint arXiv:2506.01844*, 2025.
- [40] Junjie Wen, Yichen Zhu, Jinming Li, Minjie Zhu, Zhibin Tang, Kun Wu, Zhiyuan Xu, Ning Liu, Ran Cheng, Chaomin Shen, et al. Tinyvla: Towards fast, data-efficient vision-language-action models for robotic manipulation. *IEEE Robotics and Automation Letters*, 2025.
- [41] Gi-Cheon Kang, Junghyun Kim, Kyuhwan Shim, Jun Ki Lee, and Byoung-Tak Zhang. Clip-rt: Learning language-conditioned robotic policies from natural language supervision. *arXiv preprint arXiv:2411.00508*, 2024.
- [42] Jose Barreiros, Andrew Beaulieu, Aditya Bhat, Rick Cory, Eric Cousineau, Hongkai Dai, Ching-Hsin Fang, Kunimatsu Hashimoto, Muhammad Zubair Irshad, Masha Itkina, et al. A careful examination of large behavior models for multitask dexterous manipulation. *arXiv preprint arXiv:2507.05331*, 2025.
- [43] Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Robert Equi, Chelsea Finn, Niccolo Fusai, Manuel Y Galliker, et al. $\pi_{0.5}$: a vision-language-action model with open-world generalization. In *9th Annual Conference on Robot Learning*, 2025.
- [44] Yingdong Hu, Fanqi Lin, Pingyue Sheng, Chuan Wen, Jiacheng You, and Yang Gao. Data scaling laws in imitation learning for robotic manipulation. *arXiv preprint arXiv:2410.18647*, 2024.
- [45] Yuheng Ji, Huajie Tan, Jiayu Shi, Xiaoshuai Hao, Yuan Zhang, Hengyuan Zhang, Pengwei Wang, Mengdi Zhao, Yao Mu, Pengju An, et al. Robobrain: A unified brain model for robotic manipulation from abstract to concrete. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 1724–1734, 2025.
- [46] Fanqi Lin, Ruiqian Nai, Yingdong Hu, Jiacheng You, Junming Zhao, and Yang Gao. OnetwoVLA: A unified vision-language-action model

- with adaptive reasoning. In *The Fourteenth International Conference on Learning Representations*, 2026.
- [47] Yi Yang, Jiaxuan Sun, Siqi Kou, Yihan Wang, and Zhijie Deng. Lohovla: A unified vision-language-action model for long-horizon embodied tasks. *arXiv preprint arXiv:2506.00411*, 2025.
 - [48] Michał Zawalski, William Chen, Karl Pertsch, Oier Mees, Chelsea Finn, and Sergey Levine. Robotic control via embodied chain-of-thought reasoning. *arXiv preprint arXiv:2407.08693*, 2024.
 - [49] Qingqing Zhao, Yao Lu, Moo Jin Kim, Zipeng Fu, Zhuoyang Zhang, Yecheng Wu, Zhaoshuo Li, Qianli Ma, Song Han, Chelsea Finn, et al. Cot-vla: Visual chain-of-thought reasoning for vision-language-action models. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 1702–1713, 2025.
 - [50] Zijun Lin, Jiafei Duan, Haoquan Fang, Dieter Fox, Ranjay Krishna, Cheston Tan, and Bihan Wen. Failsafe: Reasoning and recovery from failures in vision-language-action models. *arXiv preprint arXiv:2510.01642*, 2025.
 - [51] Chenyv Liu, Wentao Tan, Lei Zhu, Fengling Li, Jingjing Li, Guoli Yang, and Heng Tao Shen. Self-correcting vla: Online action refinement via sparse world imagination. *arXiv preprint arXiv:2602.21633*, 2026.
 - [52] Jacky Kwok, Christopher Agia, Rohan Sinha, Matt Foutter, Shulu Li, Ion Stoica, Azalia Mirhoseini, and Marco Pavone. Robomongo: Scaling test-time sampling and verification for vision-language-action models. In *Conference on Robot Learning*, pages 3200–3217. PMLR, 2025.
 - [53] Suhyeok Jang, Dongyoung Kim, Changyeon Kim, Youngsuk Kim, and Jinwoo Shin. Verifier-free test-time sampling for vision language action models. *arXiv preprint arXiv:2510.05681*, 2025.

- [54] Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui Chen, Christopher Dewan, Mona Diab, Xian Li, Xi Victoria Lin, et al. Opt: Open pre-trained transformer language models. *arXiv preprint arXiv:2205.01068*, 2022.
- [55] Erik Nijkamp, Bo Pang, Hiroaki Hayashi, Lifu Tu, Huan Wang, Yingbo Zhou, Silvio Savarese, and Caiming Xiong. Codegen: An open large language model for code with multi-turn program synthesis. In *The Eleventh International Conference on Learning Representations*, 2023.
- [56] Andrew N Meltzoff. Understanding the intentions of others: re-enactment of intended acts by 18-month-old children. *Developmental psychology*, 31(5):838, 1995.
- [57] Athanasios Krontiris and Kostas E Bekris. Dealing with difficult instances of object rearrangement. In *Robotics: Science and Systems*, volume 1123, 2015.
- [58] Daejong Jin, Juhan Park, and Kyungjae Lee. Perturbation-based best arm identification for efficient task planning with monte-carlo tree search. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*, pages 5758–5764. IEEE, 2023.
- [59] Zhisong Zhang, Emma Strubell, and Eduard Hovy. A survey of active learning for natural language processing. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 6166–6190, 2022.
- [60] Nils Reimers and Iryna Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 3982–3992, 2019.
- [61] Anil K Jain and Richard C Dubes. *Algorithms for clustering data*. Prentice-Hall, Inc., 1988.

- [62] Yu-Min Tseng, Yu-Chao Huang, Teng-Yun Hsiao, Wei-Lin Chen, Chao-Wei Huang, Yu Meng, and Yun-Nung Chen. Two tales of persona in llms: A survey of role-playing and personalization. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 16612–16631, 2024.
- [63] Martin Pawelczyk, Seth Neel, and Himabindu Lakkaraju. In-context unlearning: language models as few-shot unlearners. In *Proceedings of the 41st International Conference on Machine Learning*, pages 40034–40050, 2024.
- [64] OpenAI. Gpt-4o mini: advancing cost-efficient intelligence. 2024.
- [65] Google. Gemini models, 2025.
- [66] Edwin Olson. Apriltag: A robust and flexible visual fiducial system. In *2011 IEEE international conference on robotics and automation*, pages 3400–3407. IEEE, 2011.
- [67] Sachin Chitta, Ioan Sucan, and Steve Cousins. Moveit![ros topics]. *IEEE robotics & automation magazine*, 19(1):18–19, 2012.
- [68] Senyu Fei, Siyin Wang, Junhao Shi, Zihao Dai, Jikun Cai, Pengfang Qian, Li Ji, Xinzhe He, Shiduo Zhang, Zhaoye Fei, et al. Libero-plus: In-depth robustness analysis of vision-language-action models. *arXiv preprint arXiv:2510.13626*, 2025.
- [69] Harold Soh and Eugene Lim. Action hallucination in generative visual-language-action models. *arXiv preprint arXiv:2602.06339*, 2026.
- [70] Yinzhi Cao and Junfeng Yang. Towards making systems forget with machine unlearning. In *2015 IEEE symposium on security and privacy*, pages 463–480. IEEE, 2015.
- [71] Lucas Bourtole, Varun Chandrasekaran, Christopher A Choquette-Choo, Hengrui Jia, Adelin Travers, Baiwu Zhang, David Lie, and Nicolas Papernot. Machine unlearning. In *2021 IEEE symposium on security and privacy (SP)*, pages 141–159. IEEE, 2021.

- [72] Thanh Tam Nguyen, Thanh Trung Huynh, Zhao Ren, Phi Le Nguyen, Alan Wee-Chung Liew, Hongzhi Yin, and Quoc Viet Hung Nguyen. A survey of machine unlearning. *ACM Transactions on Intelligent Systems and Technology*, 16(5):1–46, 2025.
- [73] Yuanshun Yao and Xiaojun Xu. Large language model unlearning. *Advances in Neural Information Processing Systems*, 37:105425–105475, 2024.
- [74] Pratyush Maini, Zhili Feng, Avi Schwarzschild, Zachary C Lipton, and J Zico Kolter. Tofu: A task of fictitious unlearning for llms. *arXiv preprint arXiv:2401.06121*, 2024.
- [75] Robert Huben, Hoagy Cunningham, Logan Smith, Aidan Ewart, and Lee Sharkey. Sparse autoencoders find highly interpretable features in language models. In *International Conference on Learning Representations*, volume 2024, pages 7827–7845, 2024.
- [76] Leo Gao, Tom Dupre la Tour, Henk Tillman, Gabriel Goh, Rajan Troll, Alec Radford, Ilya Sutskever, Jan Leike, and Jeffrey Wu. Scaling and evaluating sparse autoencoders. In *International Conference on Learning Representations*, volume 2025, pages 26721–26754, 2025.
- [77] Alireza Makhzani and Brendan Frey. k-sparse autoencoders. In *International Conference on Learning Representations Workshop*, 2014.
- [78] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.
- [79] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. Libero: Benchmarking knowledge transfer for lifelong robot learning. *Advances in Neural Information Processing Systems*, 36:44776–44791, 2023.
- [80] Aitor Lewkowycz, Anders Andreassen, David Dohan, Ethan Dyer, Henryk Michalewski, Vinay Ramasesh, Ambrose Slone, Cem Anil,

- Imanol Schlag, Theo Gutman-Solo, et al. Solving quantitative reasoning problems with language models. *Advances in Neural Information Processing Systems*, 35:3843–3857, 2022.
- [81] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, et al. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *ACM Transactions on Information Systems*, 43(2):1–55, 2025.
- [82] Karthik Valmeekam, Alberto Olmo, Sarath Sreedharan, and Subbarao Kambhampati. Large language models still can’t plan (a benchmark for llms on planning and reasoning about change). In *NeurIPS 2022 Foundation Models for Decision Making Workshop*, 2022.
- [83] Marta Skreta, Naruki Yoshikawa, Sebastian Arellano-Rubach, Zhi Ji, Lasse Bjørn Kristensen, Kourosh Darvish, Alán Aspuru-Guzik, Florian Shkurti, and Animesh Garg. Errors are useful prompts: Instruction guided task programming with verifier-assisted iterative prompting. *arXiv preprint arXiv:2303.14100*, 2023.
- [84] Frank Joublin, Antonello Ceravola, Pavel Smirnov, Felix Ocker, Joerg Deigmoeller, Anna Belardinelli, Chao Wang, Stephan Hasler, Daniel Tanneberg, and Michael Gienger. Copal: corrective planning of robot actions with large language models. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 8664–8670. IEEE, 2024.
- [85] Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Mingwei Chang. Retrieval augmented language model pre-training. In *International conference on machine learning*, pages 3929–3938. PMLR, 2020.
- [86] Baolin Peng, Michel Galley, Pengcheng He, Hao Cheng, Yujia Xie, Yu Hu, Qiuyuan Huang, Lars Liden, Zhou Yu, Weizhu Chen, et al. Check your facts and try again: Improving large language models with external knowledge and automated feedback. *arXiv preprint arXiv:2302.12813*, 2023.

- [87] Scott Barnett, Stefanus Kurniawan, Srikanth Thudumu, Zach Bran-
nelly, and Mohamed Abdelrazek. Seven failure points when engineer-
ing a retrieval augmented generation system. In *Proceedings of the
IEEE/ACM 3rd International Conference on AI Engineering-Software
Engineering for AI*, pages 194–199, 2024.
- [88] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu
Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye,
Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback.
Advances in Neural Information Processing Systems, 36, 2024.
- [89] Chenlin Ming, Jiacheng Lin, Pangkit Fong, Han Wang, Xiaoming
Duan, and Jianping He. Hicrisp: A hierarchical closed-loop robotic
intelligent self-correction planner. *arXiv preprint arXiv:2309.12089*,
2023.
- [90] Herbert A Simon. The architecture of complexity. *Proceedings of the
American philosophical society*, 106(6):467–482, 1962.
- [91] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia,
Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting
elicits reasoning in large language models. *Advances in neural infor-
mation processing systems*, 35:24824–24837, 2022.
- [92] Katherine H Canobi. Children’s profiles of addition and subtraction
understanding. *Journal of Experimental Child Psychology*, 92(3):220–
246, 2005.
- [93] Tom Silver, Soham Dan, Kavitha Srinivas, Joshua B Tenenbaum,
Leslie Kaelbling, and Michael Katz. Generalized planning in pddl
domains with pretrained large language models. In *Proceedings of the
AAAI Conference on Artificial Intelligence*, volume 38, pages 20256–
20264, 2024.
- [94] Machel Reid, Nikolay Savinov, Denis Teplyashin, Dmitry Lepikhin,
Timothy Lillicrap, Jean-baptiste Alayrac, Radu Soricut, Angeliki

- Lazaridou, Orhan Firat, Julian Schrittwieser, et al. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. *arXiv preprint arXiv:2403.05530*, 2024.
- [95] Sudeep Dasari, Frederik Ebert, Stephen Tian, Suraj Nair, Bernadette Bucher, Karl Schmeckpeper, Siddharth Singh, Sergey Levine, and Chelsea Finn. Robonet: Large-scale multi-robot learning. In *Conference on Robot Learning*, pages 885–897. PMLR, 2020.
- [96] Frederik Ebert, Yanlai Yang, Karl Schmeckpeper, Bernadette Bucher, Georgios Georgakis, Kostas Daniilidis, Chelsea Finn, and Sergey Levine. Bridge data: Boosting generalization of robotic skills with cross-domain datasets. *arXiv preprint arXiv:2109.13396*, 2021.
- [97] Alexander Khazatsky, Karl Pertsch, Suraj Nair, Ashwin Balakrishna, Sudeep Dasari, Siddharth Karamcheti, Soroush Nasiriany, Mohan Kumar Srirama, Lawrence Yunliang Chen, Kirsty Ellis, et al. Droid: A large-scale in-the-wild robot manipulation dataset. *arXiv preprint arXiv:2403.12945*, 2024.
- [98] Homer Rich Walke, Kevin Black, Tony Z Zhao, Quan Vuong, Chongyi Zheng, Philippe Hansen-Estruch, Andre Wang He, Vivek Myers, Moo Jin Kim, Max Du, et al. Bridgedata v2: A dataset for robot learning at scale. In *Conference on Robot Learning*, pages 1723–1736. PMLR, 2023.
- [99] Sungjune Kim, Gyeongrok Oh, Heeju Ko, Daehyun Ji, Dongwook Lee, Byung-Jun Lee, Sujin Jang, and Sangpil Kim. Test-time adaptation for online vision-language navigation with feedback-based reinforcement learning. In *Forty-second International Conference on Machine Learning*, 2025.
- [100] Manli Shu, Weili Nie, De-An Huang, Zhiding Yu, Tom Goldstein, Anima Anandkumar, and Chaowei Xiao. Test-time prompt tuning for zero-shot generalization in vision-language models. *Advances in Neural Information Processing Systems*, 35:14274–14289, 2022.

- [101] Dequan Wang, Evan Shelhamer, Shaoteng Liu, Bruno Olshausen, and Trevor Darrell. Tent: Fully test-time adaptation by entropy minimization. *arXiv preprint arXiv:2006.10726*, 2020.
- [102] Chun-Mei Feng, Kai Yu, Yong Liu, Salman Khan, and Wangmeng Zuo. Diverse data augmentation with diffusions for effective test-time prompt tuning. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 2704–2714, 2023.
- [103] Yuejiang Liu, Parth Kothari, Bastien Van Delft, Baptiste Bellot-Gurlet, Taylor Mordan, and Alexandre Alahi. Ttt++: When does self-supervised test-time training fail or thrive? *Advances in Neural Information Processing Systems*, 34:21808–21820, 2021.
- [104] Chaithanya Kumar Mummadi, Robin Huttmacher, Kilian Rambach, Evgeny Levinkov, Thomas Brox, and Jan Hendrik Metzen. Test-time adaptation to distribution shift by confidence maximization and input transformation. *arXiv preprint arXiv:2106.14999*, 2021.
- [105] Zachary Nado, Shreyas Padhy, D Sculley, Alexander D’Amour, Balaji Lakshminarayanan, and Jasper Snoek. Evaluating prediction-time batch normalization for robustness under covariate shift. *arXiv preprint arXiv:2006.10963*, 2020.
- [106] Junyu Gao, Xuan Yao, and Changsheng Xu. Fast-slow test-time adaptation for online vision-and-language navigation. In *Proceedings of the 41st International Conference on Machine Learning*, pages 14902–14919, 2024.
- [107] Jian Liang, Ran He, and Tieniu Tan. A comprehensive survey on test-time adaptation under distribution shifts. *International Journal of Computer Vision*, 133(1):31–64, 2025.
- [108] Zehao Xiao and Cees GM Snoek. Beyond model adaptation at test time: A survey. *arXiv preprint arXiv:2411.03687*, 2024.
- [109] Steffen Schneider, Evgenia Rusak, Luisa Eck, Oliver Bringmann, Wieland Brendel, and Matthias Bethge. Improving robustness against

- common corruptions by covariate shift adaptation. *Advances in neural information processing systems*, 33:11539–11551, 2020.
- [110] Yu Sun, Xiaolong Wang, Zhuang Liu, John Miller, Alexei Efros, and Moritz Hardt. Test-time training with self-supervision for generalization under distribution shifts. In *International conference on machine learning*, pages 9229–9248. PMLR, 2020.
 - [111] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *The Eleventh International Conference on Learning Representations*, 2023.
 - [112] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. *Advances in neural information processing systems*, 36:11809–11822, 2023.
 - [113] Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling llm test-time compute optimally can be more effective than scaling parameters for reasoning. In *International Conference on Learning Representations*, volume 2025, pages 10131–10165, 2025.
 - [114] Qiyuan Zhang, Fuyuan Lyu, Zexu Sun, Lei Wang, Weixu Zhang, Wenyue Hua, Haolun Wu, Zhihan Guo, Yufei Wang, Niklas Muenighoff, et al. A survey on test-time scaling in large language models: What, how, where, and how well? *arXiv preprint arXiv:2503.24235*, 2025.
 - [115] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. Learning transferable visual models from natural language supervision. In *International conference on machine learning*, pages 8748–8763. PmLR, 2021.
 - [116] Basura Fernando, Amaury Habrard, Marc Sebban, and Tinne Tuytelaars. Unsupervised visual domain adaptation using subspace align-

- ment. In *Proceedings of the IEEE international conference on computer vision*, pages 2960–2967, 2013.
- [117] Junbum Cha, Wooyoung Kang, Jonghwan Mun, and Byungseok Roh. Honeybee: Locality-enhanced projector for multimodal llm. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 13817–13827, 2024.
- [118] Yuexiang Zhai, Shengbang Tong, Xiao Li, Mu Cai, Qing Qu, Yong Jae Lee, and Yi Ma. Investigating the catastrophic forgetting in multimodal large language models. *arXiv preprint arXiv:2309.10313*, 2023.

국 문 초 록

이 지 호

컴퓨터공학과 빅데이터 전공

중앙대학교 대학원

최근 로봇 학습은 대규모 데이터와 언어 기반 표현을 통해 학습된 파운데이션 모델의 발전에 힘입어 고수준 작업 계획과 저수준 행동 정책 모두에서 높은 일반화 성능을 보여주고 있다. 그러나 실제 환경에서의 로봇 운용은 사전학습된 지식의 일반화 능력만으로는 충분하지 않다. 로봇은 지각 조건, 상호작용 동역학, 작업 요구사항이 지속적으로 변화하는 동적인 물리 환경에서 동작해야 한다. 이러한 환경에서는 파운데이션 모델에 내재된 사전학습 지식이 항상 현재 상황에 적합한 로봇 행동으로 이어지지 않는다. 본 학위논문은 이러한 한계를 지식-행동 불일치 문제로 바라보며, 이는 파운데이션 모델에 내재된 지식이 신뢰할 수 있고, 실행 가능하며, 맥락에 적합한 로봇 행동을 생성하지 못하는 경우를 의미한다.

이러한 불일치 문제를 완화하기 위해, 본 학위논문은 파운데이션 모델 기반의 적응형 로봇 학습 프레임워크를 제안한다. 핵심 아이디어는 로봇이 사전학습된 지식에 단순히 의존하는 것이 아니라, 변화하는 운용 조건에서 발생하는 불일치의 원인에 따라 해당 지식을 적응시킴으로써 자신의 행동을 지속적으로 개선해야 한다는 것이다. 구체적으로 본 학위논문은 지식이 불충분할 때 경험으로부터 유용한 지식을 획득하고, 신뢰할 수 없는 행동을 유발하는 유해한 지식을 제거하며, 불확실한 지식을 정제하고, 상황 의존적 지식을 현재 맥락에 맞게 정렬하는 네 가지 상호보완적인 지식 적응 방향에 초점을 둔다. 이러한 네 가지 지식 적응 방향을 로봇 행동 생성에 직접적으로 관여하는 고수준 작업 계획과 저수준 행동 정책 수준에서 구체화한다.

고수준 작업 계획 수준에서는 대규모 언어 모델 기반 작업 계획기를 대상으로 능동적 지식 획득과 자기교정적 지식 정제 방법을 개발한다. 대규모 언어 모델 기반 계획기는 강력한 추론 능력을 제공하지만, 사전학습된 지식에 의존하기 때문에 새로운 경험을 학습에 반영하거나 잘못된 추론을 수정하는 데 한계가 있다. 불충분한 지식을 다루기 위해, 대조적 맥락 내 학습 기반의 능동적 지식 획득 방법은 능동 메모리에 저장된 성공 및 실패 경험을 대비시킴으로써 모델이 경험으로부터 유용한 지식을 획득할 수 있도록 한다. 불확실한 지식을 다루기 위해, 피드백 기반 역추론을 통한 자기교정적 지식 정제 방법은 역방향 검증을 통해 모델이 자신의 추론 과정을 검증, 분석, 수정할 수 있도록 한다.

저수준 행동 정책 수준에서는 시각-언어-행동 모델을 대상으로 선택적 지식 제거와 환경 인식 기반 지식 정렬 방법을 개발한다. 시각-언어-행동 모델은 언어 지시를 시각 관측 및 로봇 행동과 연결함으로써 높은 일반화 성능을 제공하지만, 보지 못한 시각적 또는 물리적 조건에서는 여전히 신뢰할 수 없는 행동을 생성할 수 있다. 유해한 지식을 다루기 위해, 정책 비학습 기반의 선택적 지식 제거 방법은 바람직하지 않은 행동과 관련된 내부 활성화 패턴을 억제하는 동시에 성공적인 작업 수행에 필요한 표현을 보존한다. 상황 의존적 지식을 다루기 위해, 테스트 시점 적응 기반의 환경 인식 지식 정렬 방법은 테스트 시점에 수집된 궤적을 활용하여 운용 환경 변화로 인해 흐트러진 시각 표현과 행동 표현을 재정렬한다.

종합적으로, 본 학위논문은 파운데이션 모델 기반 로봇 학습에서 발생하는 지식-행동 불일치 문제를 완화하기 위한 적응형 로봇 학습 프레임워크를 제시한다. 로봇이 경험으로부터 유용한 지식을 획득하고, 신뢰할 수 없는 행동을 유발하는 지식을 제거하며, 자신의 잘못된 결정을 정제하고, 보지 못한 조건에 맞게 행동을 정렬할 수 있도록 함으로써, 본 연구는 동적인 실제 환경에서 지속적으로 개선되고, 신뢰성 있게 일반화하며, 강건하게 동작할 수 있는 로봇 시스템을 향한 기반을 제공한다.

핵심어: 로봇 학습, 파운데이션 모델, 대규모 언어 모델, 시각-언어-행동 모델, 능동 학습, 기계 비학습, 자기교정, 테스트 시점 적응